

One Run Is Not Enough: How Much AI Agent Results Vary in Data Science, an XGBoost Optimization Study

Szilard Pafka
Epoch
szilard@epoch.com

Eduardo Ariño de la Rubia
Central European University
rubiae@ceu.edu

October 2026

Abstract

AI coding agents can automate much of a data scientist’s trial-and-error work, but their runs are not deterministic: the same agent given the same task twice tries different ideas, in a different order, and ends with a different model. We measure how much that matters. An agent (OpenAI’s Codex) was given one hour, identical instructions and a fresh isolated container to improve a starter XGBoost model that predicts flight delays, trained on 2005 flights and evaluated on 2006 flights, with every kept model scored afterwards on a holdout set the agent never sees. We ran it 20 times with each of three large language models (LLMs: gpt-6-luna, gpt-6-sol and gpt-6-astra) at the reasoning-effort setting named “max”, 60 runs in all, under automatic integrity checks; no run had to be excluded. Every run improved the starter model, and the three LLMs rank clearly on average (holdout AUC 0.6806, 0.6842 and 0.6866, with disjoint confidence intervals). But the spread within an LLM, 0.007 to 0.009 AUC from its worst run to its best, exceeds the 0.006 between the best and the worst LLM’s means, the ranges overlap, and one run of each ranks the two closest LLMs wrongly one time in five. We show how the variation builds up over the hour, relate part of the gap between LLMs to how they handle the calendar, and conclude that comparisons of agents, LLMs or prompts need repeated runs and distributions, while a better model can be obtained by running the agent several times, choosing the run with the best evaluation score, and confirming it on untouched data. The single-run building block and the multi-run orchestrator are open source.

1 Introduction

In an earlier project we showed that AI coding agents can automate much of a data scientist’s trial-and-error work (Pafka and Ariño de la Rubia, 2026a): given a working XGBoost model and a time budget, an agent researches ideas on the web, engineers features, tunes hyperparameters, and keeps what improves the model on an evaluation set. The results we presented came from single runs of a single agent. AI agents are not deterministic, though. Given the same task twice, the same agent tries different ideas, in a different order, and ends up with a different model. A single run is therefore one sample from a range of possible outcomes, and on its own it says little about how good a setup is, or whether one large language model (LLM) is better than another.

That LLM outputs vary between identical calls is well documented, down to the batch-size dependence of inference kernels (Ouyang et al., 2025; Atil et al., 2025; He and Thinking Machines Lab, 2025), and so is the variation between repeated runs of agents on benchmarks of pass-or-fail tasks (Yao et al., 2025; Bjarnason et al., 2026; Rabanser et al., 2026). What a data scientist deciding between

agents, LLMs or prompts needs is something more specific: the size of that variation on a continuous measure of the model an agent delivers, on a realistic task, under a protocol that rules out shortcuts, and for LLMs of clearly different strength, so that the spread within an LLM can be set against the gaps between LLMs. This paper measures that.

We rebuilt the earlier project into a minimal, self-contained building block for one autonomous run and an orchestrator that runs it many times without any human in the loop. The task is to improve a starter XGBoost model that predicts whether a flight will depart 15 minutes or more late, trained on flights from 2005 and evaluated on flights from 2006. The agent has one hour. Every model it keeps is then scored on a holdout set of other 2006 flights that the agent never sees. Each run starts in a fresh, isolated container with identical instructions; the holdout data and the scoring scripts are out of the agent’s reach until it has finished; and automatic checks afterwards verify that the agent followed the rules. With this we ran OpenAI’s Codex agent 20 times with each of three OpenAI LLMs, gpt-6-luna, gpt-6-sol and gpt-6-astra (Luna, Sol and Astra for short), all at the reasoning-effort setting named “max”: 60 runs, 2,446 experiments, none excluded.

The paper makes five contributions.

1. A minimal, standardized, open building block for one autonomous run (one editable file, a harness that keeps the clock, times and kills experiments, saves an artifact per commit and scores the evaluation set row by row), and an orchestrator that runs it repeatedly without human steering, with per-run isolation, automatic integrity and protocol checks, holdout scoring and archived session logs. Both are open source, so others can test their own agents, LLMs and setups, with more than one run each.
2. Sixty fully automated, automatically checked and reviewed runs: three LLMs of one frontier family, 20 runs each, same agent, same prompt, same image, same machine type.
3. Measurements of the run-to-run variation on a continuous outcome, the holdout AUC of the delivered model: the distribution per LLM, the probability that a single run of each LLM ranks two LLMs correctly, and the time course of the holdout AUC within the hour.
4. A partial account of where the differences come from: which modeling decisions separate the LLMs and the runs, in particular how the calendar is handled, and the role of luck.
5. Practical guidance on how to compare agents, LLMs or prompts, and on how to get a good model from an agent whose runs vary.

The findings are these. Every one of the 60 runs improved the starter model on the holdout set, and on average the three LLMs rank clearly: holdout AUC 0.6806 for Luna, 0.6842 for Sol and 0.6866 for Astra, with confidence intervals that do not overlap. But the runs of each LLM are spread widely. From worst to best run, each LLM spans 0.007 to 0.009 AUC, more than half of its average improvement over the starter model and more than the 0.006 between the best and the worst LLM’s means, and the ranges overlap. The standard deviation between runs of one LLM, about 0.002, is close to the 0.0024 between the means of the two closest LLMs. One run of each LLM ranks Astra above Luna 98% of the time and Sol above Luna 91% of the time, but Astra above Sol only 80% of the time. The variation builds up over the hour: most runs make the same early move, a few minutes into the hour for Astra and Sol and later for Luna, and then diverge, with Astra’s advantage over Sol opening mostly in the second half hour. Part of the gap between the LLMs is associated with one decision, whether to keep the day of the month as a category, which Astra almost always dropped and Luna mostly kept. Two practical consequences follow: comparisons of agents, LLMs, prompts or any other part of a setup need repeated runs and distributions, not single numbers; and when the goal is a good model, it pays to run the agent several times, choose the run with the best evaluation score, and confirm the chosen model on data that nobody has touched.

Section 2 places the study in prior work. Section 3 describes the task, the harness, the orchestrator, the agent and the LLMs, the checks and the statistics. Section 4 gives the results, Section 5 discusses what they mean and what they do not, and Section 6 concludes. The appendices list every run, the validity checks, the rules given to the agent, the statistical details, the feature audit, the software and dates, one run in detail, and the token usage.

2 Related work

2.1 Agents that do machine-learning engineering

The loop in which an agent edits a training script, runs it, keeps the change if a metric improves and reverts it otherwise, guided by a Markdown file of instructions, was popularized by Karpathy’s autoresearch, which applies it to the training of a small language model under a fixed five-minute budget per experiment (Karpathy, 2026). Our earlier project adapted it to XGBoost on tabular data (Pafka and Ariño de la Rubia, 2026a); the single-run repository of this paper is a simplified and standardized descendant of it. Ferreira et al. (2026) use autoresearch as a testbed to compare LLM agents with classical hyperparameter optimization under a fixed budget, and find that letting the agent edit the code narrows the gap to the classical methods without closing it; on tabular benchmarks, a budget-matched, multi-seed comparison found an LLM advisor added nothing measurable on held-out data once classical search started from the same default (Rodrigues et al., 2026). Whether agents beat classical search is a different question from ours, which compares agents with each other and single runs with repeated ones. LLMs have also been used directly as hyperparameter proposers, inside Bayesian optimization, and as agents that run the tuning loop (Zhang et al., 2023; Liu et al., 2024b,a); our agent tunes by editing code, which lets it engineer features as well as set parameters.

A growing set of benchmarks evaluates agents on machine-learning engineering and data-science tasks: MAgentBench on 13 experimentation tasks (Huang et al., 2024), MLE-bench on 75 Kaggle competitions (Chan et al., 2025), RE-Bench on seven open-ended research-engineering environments (Wijk et al., 2025), DSbench on 540 analysis and modeling tasks (Jing et al., 2025), DeltaML-Bench on 48 tasks drawn from research repositories (Moukpe et al., 2026), and DSAgentBench on 275 end-to-end workflows in real computer environments (Rahman et al., 2026), with agents built for them such as DS-Agent, AIDE, AutoKaggle, Agent K and MLE-STAR (Guo et al., 2024; Jiang et al., 2025; Li et al., 2024; Grosnit et al., 2024; Nam et al., 2025). These efforts cover far more tasks than ours. Two of them speak directly to repetition: MLE-bench’s evaluation guidance asks for at least three seeds because agents are high-variance, and RE-Bench aggregates repeated attempts as best-of-k under different time budgets. We study one task deeply enough to estimate the distribution of repeated identical attempts, and to see how it builds up within a run.

2.2 Variation between identical runs

That a language model answers the same prompt differently on repeated calls, even at temperature zero, has been measured for code generation and for classification (Ouyang et al., 2025; Atil et al., 2025), and traced to inference kernels whose results depend on how requests are batched (He and Thinking Machines Lab, 2025). An agent compounds hundreds of such calls, each conditioned on the outcomes of the earlier ones, so its runs diverge early and end in different places (Mehta, 2026). Agent benchmarks have begun to measure this. τ -bench’s pass^k, the share of tasks an agent solves in all of k trials, falls well below pass¹, the share solved in a single trial (Yao et al., 2025). Rabanser et al. (2026) argue that a single success rate hides consistency, robustness and predictability.

Bjarnason et al. (2026) ran six coding-agent configurations ten times each on SWE-bench Verified. Single-run pass@1, the share of tasks solved in one attempt, moved by several points depending on which run one looked at, with standard deviations above 1.5 points even at temperature zero. They recommend several runs per task, a power analysis to choose their number, and reporting pass@k, the share solved in at least one of k attempts, and pass^k beside pass@1. Their unit is a pass rate over hundreds of tasks; ours is a continuous score of the delivered model on one task, which lets us ask how far apart two runs of the same setup end up, and how often one run of each ranks two setups wrongly.

The variance of evaluation benchmarks themselves, from seeds, prompts and item sampling, has been quantified for language models (Madaan et al., 2024; Wang, 2025), and the statistics of comparing noisy systems (error bars, paired comparisons and sample-size planning) have been set out for them (Miller, 2024), with the warning that normal-approximation intervals are optimistic when the number of data points is small (Bowyer et al., 2025) and with Bayesian alternatives to pass@k (Hariri et al., 2026). Cost belongs in the comparison too, and agents should be run several times before being compared (Kapoor et al., 2025). All of this has an older history in machine learning and reinforcement learning, where nondeterminism and few runs per method made reported improvements hard to interpret (Henderson et al., 2018), the number of seeds needed for a given power was worked out (Colas et al., 2018), interval estimates and robust aggregates became the recommended practice (Agarwal et al., 2021), variance from data sampling, initialization and hyperparameters was shown to change benchmark conclusions (Bouthillier et al., 2021), and fine-tuning one model 2,100 times with different seeds showed how much the best observed result depends on how many trials were run (Dodge et al., 2020). Our runs are the agent analogue of seeds, with one difference that matters for practice: nothing in the setup can be fixed to make two runs identical.

2.3 Comparing noisy systems and selecting among attempts

The head-to-head statistic we report, the probability that a random run of one LLM beats a random run of another, is the Mann-Whitney probability of superiority, proposed as a common-language effect size and refined to count ties half (Mann and Whitney, 1947; McGraw and Wong, 1992; Vargha and Delaney, 2000); its intervals come from the bootstrap (Efron, 1979). Repeated attempts have a second use besides measurement: they raise the best result one can select. Repeated sampling raised the share of programming problems solved by early code models (Chen et al., 2021), and its gains scale over orders of magnitude of samples (Brown et al., 2024), and best-of-k is an established way to score agents on research-engineering tasks (Wijk et al., 2025). Selecting the best of several candidates on a set and reporting that set’s score overstates the chosen candidate’s quality (Cawley and Talbot, 2010; Dwork et al., 2015b,a); a holdout kept out of the selection avoids this, and in practice the optimism from repeated use of a holdout has often been found small (Roelofs et al., 2019). We keep selection and measurement apart: the agent selects on the evaluation set, the holdout set measures, and we report what selecting among runs on the evaluation set is worth on the holdout set.

2.4 Agents as deployed systems, and rule-following

An agent is a model inside an agent harness, the software that carries out the model’s tool calls and manages its context, and the two interact: changing only the agent harness changes what a fixed model achieves (Lewis, 2026), the effect of its components depends on the model (Fan et al., 2026), and it moves cost as much as success (Pan et al., 2026). We hold the agent harness fixed

and vary the LLM; following common usage, the rest of the paper calls the agent harness, Codex, the agent, so what we compare are three deployed agent-LLM-service systems; a companion study by the same authors also uses repeated runs of coding agents on an XGBoost tuning task, with open-weight models and several agent harnesses (Ariño de la Rubia and Pafka, 2026), and the claims of the present paper rest only on the 60 runs reported here. Capable agents sometimes break the rules of a task: reward hacking has been found in frontier-model evaluations (METR, 2025), gaps between visible and held-out tests grow with task size (Zhao et al., 2026), agents under pressure exploit labeled public evaluation files (Chen et al., 2026), ML agents have been observed gaming specifications (Moukpe et al., 2026), and flaws in task setup or scoring can misestimate agent performance badly (Zhu et al., 2025). This is why our protocol keeps the holdout set and the scoring scripts out of the agent’s reach, scores the evaluation set one row at a time so that batch-dependent features cannot work, checks every commit, artifact and timing row, and searches the session log for the content of the forbidden files. None of the 60 runs broke the data rules; the machinery is a methods contribution, its integrity flags were all false alarms on review, and the protocol caveats it raised are reported with the runs.

2.5 Tabular learning and temporal shift

Gradient-boosted trees remain the strongest default on tabular data of this size (Grinsztajn et al., 2022; Shwartz-Ziv and Armon, 2022), and XGBoost (Chen and Guestrin, 2016) is the implementation the agent tunes. Time-based splits change which methods and settings look best on tabular data (Rubachev et al., 2025; Gardner et al., 2023; Cai and Ye, 2025), and our task is built on one: the agent trains on 2005 and is evaluated on 2006, so a model that memorizes particular days of the training year is punished on the evaluation set, which is what the calendar finding of Section 4.5 is about. The data, described in Section 3.1, have long served for benchmarking machine-learning software (Pafka, 2015).

2.6 Positioning

Run-to-run variation of LLM agents, the need for repeated runs and power analysis, best-of-k selection, and the sensitivity of agents to their agent harness are each documented. What this paper adds is a measurement a practicing data scientist can use: on one realistic tabular task, with one agent and three LLMs of one frontier family at a fixed effort, 20 identical, automatically checked and reviewed runs each, how wide the distribution of the delivered model’s holdout AUC is, how it compares with the gaps between LLMs, how often a single run of each ranks two LLMs wrongly, how the variation arises over the hour, and which modeling decisions account for part of it. The building block and the orchestrator make the same measurement repeatable for other agents, LLMs and prompts.

3 Methods

3.1 Task and data

The task is the second scenario of our earlier project (Pafka and Ariño de la Rubia, 2026a): predict whether a US domestic flight departs 15 minutes or more late, from what is known before departure. The data are the airline on-time records of the ASA Data Expo 2009 (Data Expo 2009), which one of us has used for benchmarking machine-learning software since 2015 (Pafka, 2015). A human-only script (`human/make_data.py`) reads the 2005 and 2006 files, keeps the flights that departed, labels

a flight as delayed when its departure delay is 15 minutes or more, and keeps eight predictors: month, day of month and day of week (as categorical strings), scheduled departure time and distance (numeric), and carrier, origin airport and destination airport. It writes three files with fixed random seeds, so every run sees identical data: a training set of 200,000 flights sampled from 2005, 100,000 per class; and an evaluation set and a holdout set of 50,000 flights each, the two disjoint halves of one balanced sample of 100,000 flights from 2006.

The metric is AUC, the area under the receiver operating characteristic (ROC) curve. The agent optimizes the AUC on the evaluation set, which we call the eval AUC. After the run, every model the agent kept is scored on the holdout set, which the agent never sees; we call that the holdout AUC. With 25,000 flights per class, the standard error of one AUC near 0.68 is about 0.002 (Hanley and McNeil, 1982). That is the uncertainty about how a model would score on other flights of 2006, and it is about as large as the spread between runs reported below. But every model is scored on the same holdout rows, so most of that sampling error is shared, and the differences between models, which are what this paper compares, are measured more precisely (DeLong et al., 1988).

The year shift is deliberate. A model that fits the 2005 flights more closely does not necessarily predict the 2006 flights better, and the instructions tell the agent so: “`train.csv` is sampled from 2005 flights while `eval.csv` (and the holdout set) is from 2006, so a validation split off `train.csv` overstates the AUC and can favour more complex models than the eval set does.”

3.2 The starter model and the file the agent edits

The agent edits one file, `train.py`, 47 lines in its starting form. The starter reads the training set, fixes the levels of the six categorical columns on the training data (levels unseen in training become missing values, which XGBoost handles natively), fits an XGBoost classifier (Chen and Guestrin, 2016) with 100 trees of depth 6 at learning rate 0.1 with native categorical support, and hands the fitted model and the `prepare` function to the harness. It scores 0.6743 on the evaluation set and 0.6725 on the holdout set. We call this the starter model.

Everything in the file is open to the agent: data preparation, feature engineering, hyperparameters, the model itself. One constraint shapes what features are possible. `prepare(df)` must compute each row’s features from that row alone, plus lookup tables fitted on the training data at module level, because the harness scores the evaluation set one row at a time. Features that aggregate over the rows being scored, such as counts or group means of the evaluation rows, therefore cannot work, and the instructions explain this with examples. Lookups fitted on the training data, such as a route’s median departure time, are allowed and were used by many runs.

3.3 The harness and the rules

The harness (`harness.py`, which the agent may not modify) keeps the clock and runs the experiments. `python3 harness.py start` starts a one-hour budget. `python3 harness.py run` executes `train.py`, kills it if the training phase exceeds 60 seconds or the evaluation phase 300 seconds, and refuses to start once the budget is spent. If `train.py` is committed, the harness saves the fitted model together with `prepare` as an artifact named after the commit; it then reloads that artifact and scores the evaluation set with it, row by row across all cores, and prints the eval AUC with four decimals. The starter takes about 30 seconds per experiment, most of it evaluation. `python3 harness.py stop` ends the run, and `python3 harness.py report` splits the hour into time spent in experiments and time spent by the agent on everything else: thinking, web research, editing, tool calls.

The agent’s instructions are a Markdown file, `program.md`, in the tradition of Karpathy’s autoresearch (Karpathy, 2026); the parts that set the rules are reproduced in Appendix C. The agent creates a branch, reads the two files it needs, `train.py` and `harness.py`, confirms the setup, starts the clock and then loops until the budget is spent: review the results so far and state a hypothesis; edit `train.py`; commit; run the experiment through the harness; read the eval AUC; log the experiment with a one-line description in a results table and the reasoning in a research log; keep or discard. The keep rule is strict:

Keep an experiment only if its Eval AUC, as printed by the harness (4 decimals), is higher than that of the last kept commit. If it is exactly equal, keep it only if the code is simpler or faster (a simplification or clean-up at no cost in AUC). If it is lower, discard it, however small the drop and however much simpler the code.

Discarding means resetting the branch to the last kept commit. Web research is mandatory: before the first experiment, every ten experiments, and whenever three consecutive experiments fail to move the eval AUC by 0.001. Every ten experiments the agent also writes a short synthesis of what it has learned. The agent may inspect the training data with read-only commands. It may not read, run or reference anything in the human-only folder, the holdout set, or the source files from which the data were built; it may not fetch other data, install packages, change the evaluation, or look at earlier runs’ results in git or on disk. When less than two minutes remain it wraps up, writes a final summary, and stops the clock. The instructions estimate that an experiment takes one to two minutes including the agent’s own time, so 30 to 60 experiments fit in the hour.

Throughout the paper, a *run* is one hour-long session of the agent, and an *experiment* is one timed execution of `train.py` through the harness, that is, one row of the agent’s results table, the baseline included. A *kept commit* is the baseline or an experiment kept under the keep rule, so a run with n kept commits made $n - 1$ kept changes.

3.4 The orchestrator: isolation and automation

Each run takes place in a fresh Docker container built from one image. The image holds Ubuntu 26.04, the Codex CLI installed as an unprivileged user without `sudo`, the Python packages, and a copy of the single-run repository at a pinned commit, with fresh git history and without the archived results of earlier runs, so the agent cannot see what previous runs did. The data files are built inside the image by `make_data.py`. The human-only scripts and the holdout set are moved out of the agent’s copy to a directory readable only by root, and are copied back only after the agent has exited. The only thing shared between containers is the agent’s login, through a Docker volume. Each container runs with a memory cap of 24 GiB and no swap, so a runaway experiment is killed by the kernel instead of starving the machine; the harness records such a kill as a crash or a timeout. Nothing was killed at the cap in any of the 60 runs.

A shell script drives every run identically. It starts the container, checks that the human-only files are out of the agent’s reach, runs the starter once as a setup check (expecting an eval AUC of 0.6743), verifies that the requested LLM and reasoning-effort level exist in the Codex model catalog, and then talks to the agent through `codex exec`, one command per turn, with approvals and sandboxing turned off inside the container. The first turn is the prompt from the single-run repository’s README, “Hi have a look at `program.md` and let’s kick off a new experiment! let’s do the setup first.” The agent reads the files and sets up; the driver answers “go” until the harness clock starts, and “keep going” if the agent stops while time is left. The harness enforces the hour. If a turn ends with a service error, the driver waits (30 seconds after “model at capacity”, five minutes

after anything else) and resends; the clock keeps running meanwhile, and the time lost is recorded. Once the agent has exited, the driver records the container’s peak memory, kills anything the agent left running, runs the harness report and the automatic checks of Section 3.6, copies the human-only files back, scores every kept commit on the holdout set, runs a leak check over the agent’s session log, archives the run (results table, research log, the final `train.py`, holdout scores, timing, checks, the git log, and the session log with the model’s encrypted reasoning dropped and account identifiers redacted) and deletes the container.

The orchestration itself was done by another AI coding agent, Claude Code, through a project skill that launches the driver for run after run, reviews each finished run against the checks, and writes a review file per run and a summary per group, the 20 runs of one LLM; the authors reviewed those reviews and committed the results. Every run used an AWS m8i.2xlarge instance (8 vCPUs, 32 GiB of memory). Runs within a group were sequential, since each uses all cores of its machine. The groups partly ran at the same time, on separate instances, and the starter’s evaluation took 30 to 31 seconds in every run’s setup check, so no run competed with another for the CPU.

3.5 Agent, LLMs and settings

The agent is OpenAI’s Codex CLI, version 0.160.0 (OpenAI, 2026a), used through a ChatGPT subscription. The three LLMs are OpenAI’s GPT-6 family (OpenAI, 2026c): gpt-6-luna, gpt-6-sol and gpt-6-astra, which we call Luna, Sol and Astra. Luna is the vendor’s smallest and cheapest tier, Astra its flagship, Sol in between. Every run used the reasoning-effort setting named `max`. That is Luna’s highest level. For Sol and Astra, Codex’s model catalog lists one more level, `ultra`, above it, while the vendor’s API documentation lists `max` as the highest level for all three (OpenAI, 2026c,b); the comparison in this paper is therefore between the three LLMs at one named setting, not between each LLM at its ceiling. The driver confirmed the LLM and the effort level from every session log. Web search was on, since the instructions require research.

The Luna and Sol runs were made on 5 and 6 October 2026 and the Astra runs on 6 and 7 October 2026 (Appendix F). Table 1 summarizes the setup. The LLMs are hosted services whose builds, load and routing are not under our control. What we measure is therefore the variation of the deployed system, agent, LLM and service together, over those days, as anyone using the same service would meet it.

Table 1: The setup.

item	setting
Task	predict whether a flight departs 15 minutes or more late, from month, day of month, day of week, scheduled departure time, carrier, origin, destination and distance
Data	train: 200,000 flights of 2005, balanced; eval and holdout: 50,000 flights of 2006 each, the disjoint halves of one balanced sample; Data Expo 2009 airline data, fixed seeds
Metric	AUC; the agent optimizes eval AUC; every kept model is scored on the holdout set afterwards
Starter model	XGBoost, 100 trees, depth 6, learning rate 0.1, native categoricals; eval 0.6743, holdout 0.6725
Budget and limits	1 hour of wall-clock time per run; training killed after 60 s, evaluation after 300 s; wrap up when under 2 minutes remain
Keep rule	keep only a strictly higher eval AUC (4 decimals), or a tie with simpler or faster code; otherwise reset
Agent	OpenAI Codex CLI 0.160.0 on a ChatGPT subscription; approvals and sandbox off inside the container; web search on

item	setting
LLMs	gpt-6-luna, gpt-6-sol, gpt-6-astra, all at reasoning effort max
Isolation	a fresh container per run (Ubuntu 26.04), pinned code with fresh git history, human-only scripts and the holdout set unreadable until the agent exits, 24 GiB memory cap
Machines	AWS m8i.2xlarge (8 vCPUs, 32 GiB) for every run; runs sequential within each LLM’s group, groups partly in parallel on separate instances
Runs	20 per LLM, 60 in all, 5 to 7 October 2026; 0 excluded, 10 with a protocol caveat
Checks	integrity checks, a leak check of the session log among them (exclude if a flag is confirmed on review); protocol checks (caveat); holdout scoring of every kept commit; each run reviewed

3.6 Validity checks

Two kinds of automatic checks run after every run. The integrity checks look for anything that would invalidate a result: a commit that touches any file other than `train.py` (the agent’s own logs aside); a modified harness; extra files in the data folder; an artifact or a kept result that did not come from an experiment run through the harness inside the clock; a version of `train.py` that reads any file other than the training set or accesses the network; a download of flight data from anywhere; and, through a leak check of the session log, any content of the holdout set, the evaluation set or the human-only scripts, searched by distinctive lines and data rows rather than by file names. A run whose integrity flag is confirmed on review is excluded. The protocol checks look for breaches of the agent’s own rules that do not invalidate the result: a kept commit with a lower eval AUC than the one before, a discarded commit left in the branch, kept commits out of order, the clock stopped early, outputs committed, stray files, the driver having to stop the clock, retry waits above two minutes, or an attempt to read or run a human-only file or the holdout set, which yields nothing because those files are unreadable during the run. A run that fails a protocol check stays in the analysis with a caveat. In addition, the review compares the gap between the holdout and the eval AUC of every run’s final model (defined in Section 3.7) with two thresholds fixed before the runs: a gap below -0.006 would mean that the agent overfit the evaluation set, and a gap above +0.003, a holdout score clearly higher than the eval score, would be unusual enough to send the run’s integrity checks through a second review. No run crossed either; the gaps ranged from -0.0037 to -0.0006 (Table 7).

No run was excluded. The integrity checks, the leak check included, flagged 10 of the 60 runs, and every flag turned out to be a false alarm on review: in five runs the pattern meant to catch file globbing matched the word “global” in a variable name; in one, a read of the training set was written differently from the starter’s; in two, an artifact belonged to an experiment inside the clock whose timing row was missing or mislabeled, because the turn failed during evaluation or the agent reset the repository before the evaluation had finished; and two leak checks matched text that the agent itself had produced. Ten runs carry a protocol caveat: nine kept at least one commit that only tied the best eval AUC without being simpler or faster (the agents called a change 0.2 to 1.1 seconds “faster”, within the timing noise), and one lost 216 seconds of its hour to retries while its LLM was at capacity. Table 10 in Appendix B lists them. All 60 runs are analyzed alike; Table 11 in Appendix B also shows that leaving the caveat runs out changes nothing.

Reading documentation on the web is what the agent is asked to do, and three runs read pages of the US Bureau of Transportation Statistics (BTS) that carry information about the evaluation year: two Astra runs learned that the carriers America West and US Airways reported jointly from 2006 and mapped one to the other (worth 0.0004 and 0.0007 on the holdout set when made), and one Sol run and one of those Astra runs built holiday windows from a table of holiday travel seasons by

year. No run saw flight records, and none read the evaluation set. These runs are kept; Appendix B shows the results without them.

3.7 Outcome measures and statistics

The primary outcome of a run is the holdout AUC of its *final model*, the kept model with the highest eval AUC (on a tie, the last kept commit). For each LLM we report the mean, the sample standard deviation, the minimum and maximum, the 10th and 90th percentiles (the 3rd-lowest and 3rd-highest of the 20 runs), the median, and a 95% confidence interval for the mean from the t distribution. For each pair of LLMs we report the probability that a randomly chosen run of one beats a randomly chosen run of the other, estimated as the share of all 400 pairs of runs in which it does, ties counted half: the Mann-Whitney probability of superiority, also known as the common-language effect size or the Vargha-Delaney A measure (Mann and Whitney, 1947; McGraw and Wong, 1992; Vargha and Delaney, 2000), with a 95% percentile bootstrap interval from 10,000 resamples of the runs within each LLM (Efron, 1979). Welch’s t-tests (Welch, 1947), Mann-Whitney tests and an analysis of variance are in Table 12, Appendix D.

To see how a run unfolds, we trace the holdout AUC of the kept model over the hour: each kept commit counts from the end of its experiment until the next keep, with the clock starting at `harness.py start`. The median path of an LLM at a given minute is the median over its 20 runs of the kept model’s holdout AUC at that minute, a finished run counting with its final value. To see what the agents changed, we audited the final `train.py` of every run with a script (Appendix E), checking in particular how it treats the month and the day of the month and whether it builds holiday or day-of-year features; the script flags files whose structure it may misread, and every file was then read in full and the script’s verdicts confirmed or corrected. Three further analyses were defined after the runs and are labeled as post hoc or exploratory where they appear: the value of running the agent several times and choosing the run with the best eval AUC (best of k), computed exactly over the observed runs with draws with replacement; the number of runs per arm that the usual approximation would require to detect the gaps we observed; and the relation between the number of experiments in a run and its result.

All numbers in the paper are produced by scripts in the paper’s repository from the archived runs, and every table’s source file is committed with the command that wrote it. The analyses use numpy 2.5, scipy 1.18 and matplotlib 3.11; the runs used xgboost 3.4.1, pandas 3.0.6, scikit-learn 1.9.1, numpy 2.5.3 and cloudpickle 3.1.2.

4 Results

4.1 All runs improve the starter model, and the LLMs rank on average

Figure 1 shows the holdout AUC of each run’s final model, Table 2 the statistics per LLM, and Table 9 in Appendix A every run. Every one of the 60 runs beat the starter model’s 0.6725; the weakest run, a Luna run, reached 0.6773. On average Astra improved the starter by 0.0141, Sol by 0.0117 and Luna by 0.0081, and the three LLMs rank clearly: the 95% confidence intervals of their means do not overlap, and the means differ by 0.0024 (Astra over Sol), 0.0035 (Sol over Luna) and 0.0059 (Astra over Luna).

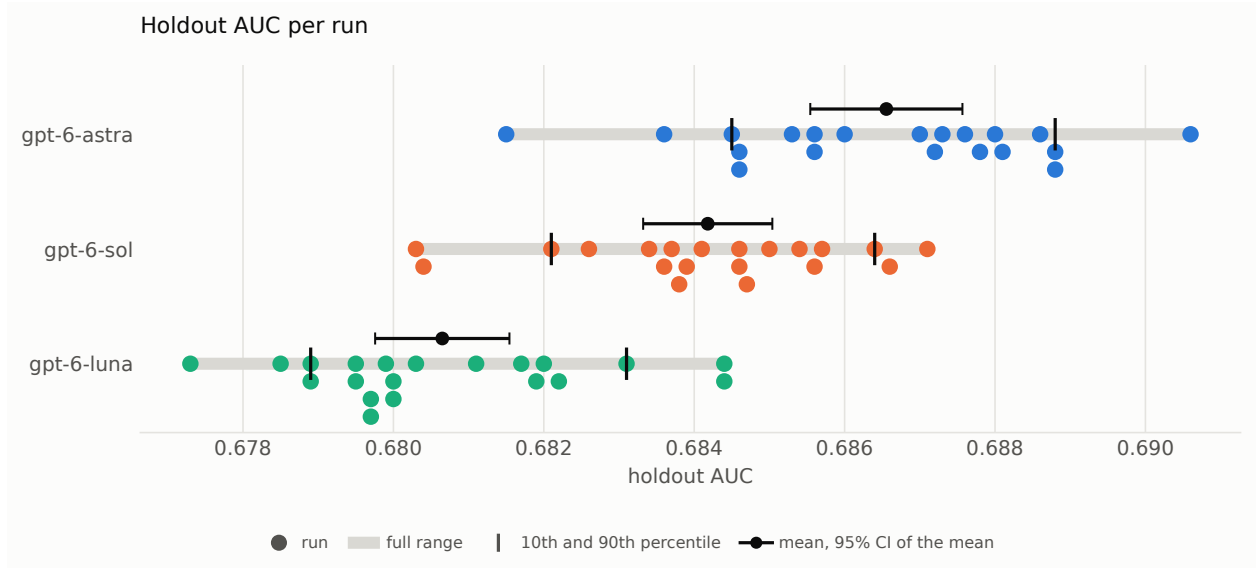


Figure 1: Holdout AUC of each run’s final model, 20 runs per LLM. Dots: runs. Gray bar: the full range. Vertical ticks: the 10th and 90th percentiles. Black dot and whiskers: the mean and its 95% confidence interval. The starter model scores 0.6725.

Table 2: Holdout AUC of each run’s final model, 20 runs per LLM. The starter model scores 0.6725. sd is the sample standard deviation; the interval uses the t distribution; the 10th and 90th percentiles are the 3rd-lowest and 3rd-highest of the 20 runs; the range is the best run minus the worst.

LLM	n	mean	sd	95% CI of the mean	min	10th pct.	median	90th pct.	max	mean gain over starter	range
Astra	20	0.6866	0.0022	0.6855–0.6876	0.6815	0.6845	0.6871	0.6888	0.6906	+0.0141	0.0091
Sol	20	0.6842	0.0018	0.6833–0.6850	0.6803	0.6821	0.6844	0.6864	0.6871	+0.0117	0.0068
Luna	20	0.6806	0.0019	0.6798–0.6815	0.6773	0.6789	0.6800	0.6831	0.6844	+0.0081	0.0071

4.2 The runs of one LLM vary as much as the LLMs differ

The runs of each LLM are spread widely. From worst to best run, Luna spans 0.0071, Sol 0.0068 and Astra 0.0091. For each LLM, that range is more than half of its average improvement over the starter model (87%, 58% and 65% of it), and larger than the 0.0059 between the means of the best and the worst LLM. The range grows with the number of runs, so a steadier measure of the spread is the standard deviation: 0.0018 to 0.0022 within an LLM, against 0.0024 between the means of the two closest LLMs. The ranges also overlap: Luna’s two best runs (0.6844) beat half of Sol’s 20 runs and the two worst of Astra’s, and Sol’s best run (0.6871) beats half of Astra’s. A single run can therefore land almost anywhere within an interval that covers more than one LLM’s typical results.

4.3 One run of each LLM often ranks two LLMs wrongly

Table 3 and Figure 2 turn the overlap into the question a practitioner faces: if I run each LLM once, how often does the better one win? Astra beats Luna in 98% of all pairs of runs and Sol beats Luna in 91%, but Astra beats Sol in only 80%: Sol still wins one pairing in five, although the two means

differ by 3.7 standard errors of their difference. These are point estimates from 20 runs per LLM, and the bootstrap intervals show how uncertain they remain even with that many runs: for Astra against Sol, from 66% to 93%. Comparing two LLMs on a single run each would therefore give the wrong answer one time in five for the closest pair here, even though one of them is clearly better on average.

Table 3: Head to head: the probability that a randomly chosen run of the first LLM has a higher holdout AUC than a randomly chosen run of the second, over all 400 pairs of runs, ties counted half. Bootstrap: runs resampled within each LLM, 10,000 resamples.

pair	P(first LLM’s run wins)	95% bootstrap interval	difference of means
Astra vs Sol	80%	66%–93%	+0.0024
Astra vs Luna	98%	93%–100%	+0.0059
Sol vs Luna	91%	81%–98%	+0.0035

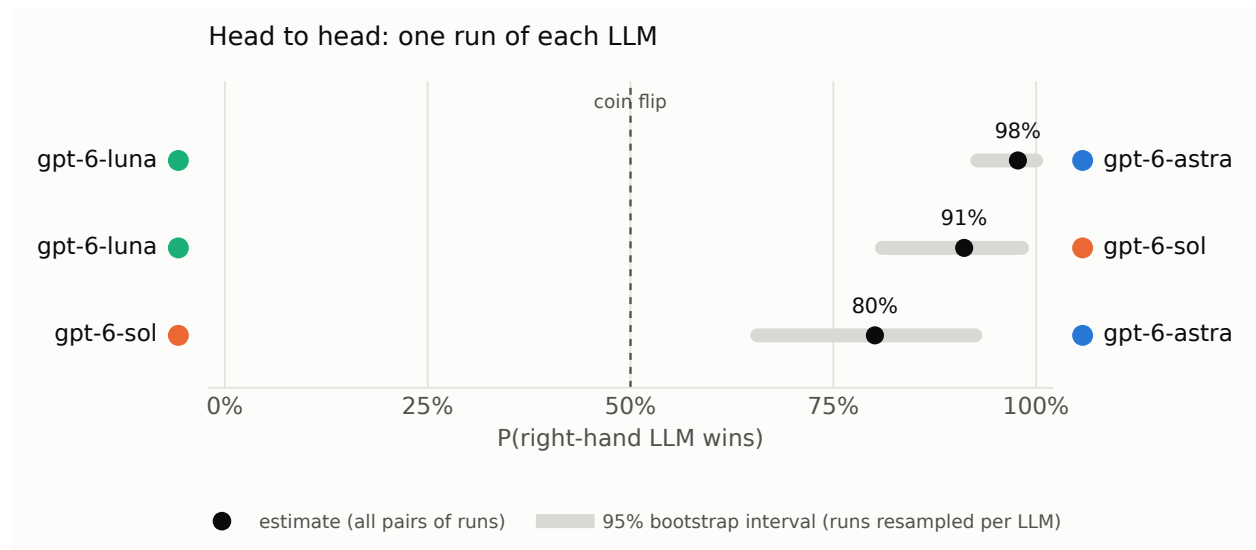


Figure 2: Head to head: the probability that a randomly chosen run of the LLM named on the right beats a randomly chosen run of the LLM named on the left, over all 400 pairs of runs, ties counted half, with 95% bootstrap intervals.

4.4 How the variation builds up over the hour

Figures 3 and 4 trace every run over its hour. Each thin line is one run and each step a change the agent kept, usually up, but sometimes down, when a gain on the evaluation set does not carry over to the holdout set. The bold lines are the medians of each LLM’s runs. Table 4 summarizes the hour and Table 5 the first moves; Appendix G follows one run commit by commit.

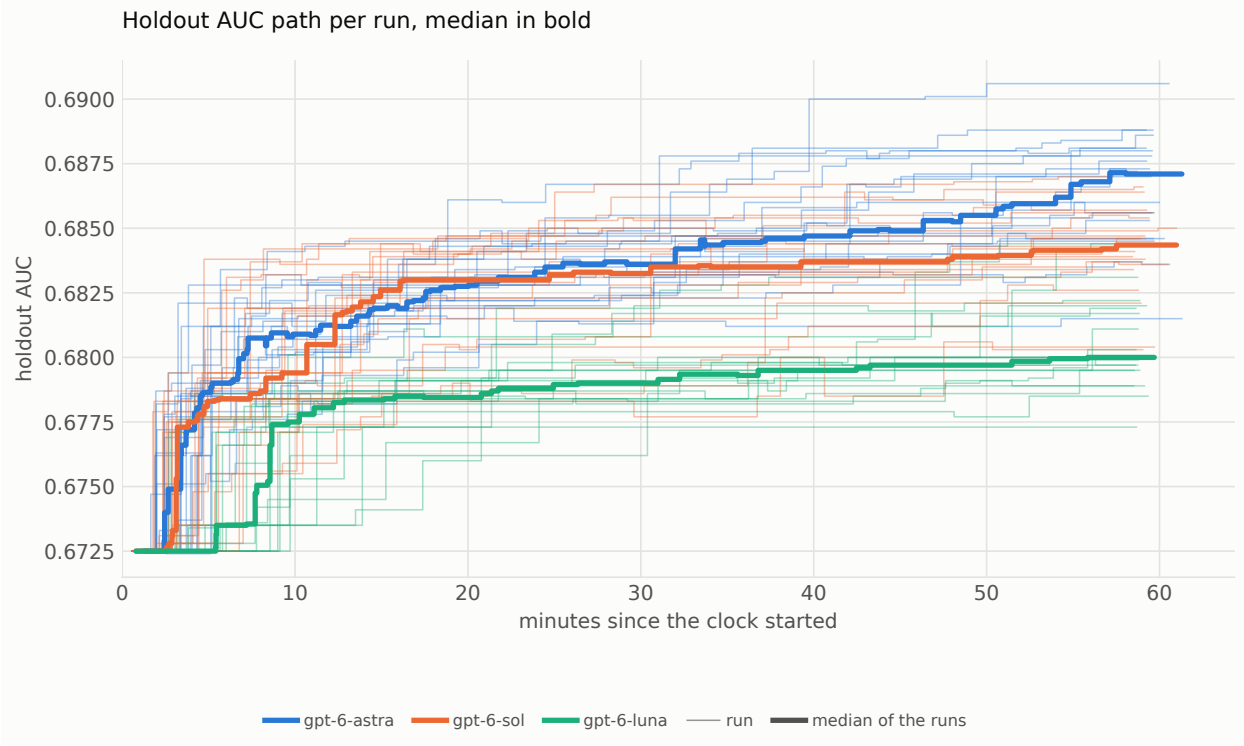


Figure 3: Holdout AUC of the kept model over the hour, one thin line per run, with the median over each LLM’s runs in bold.

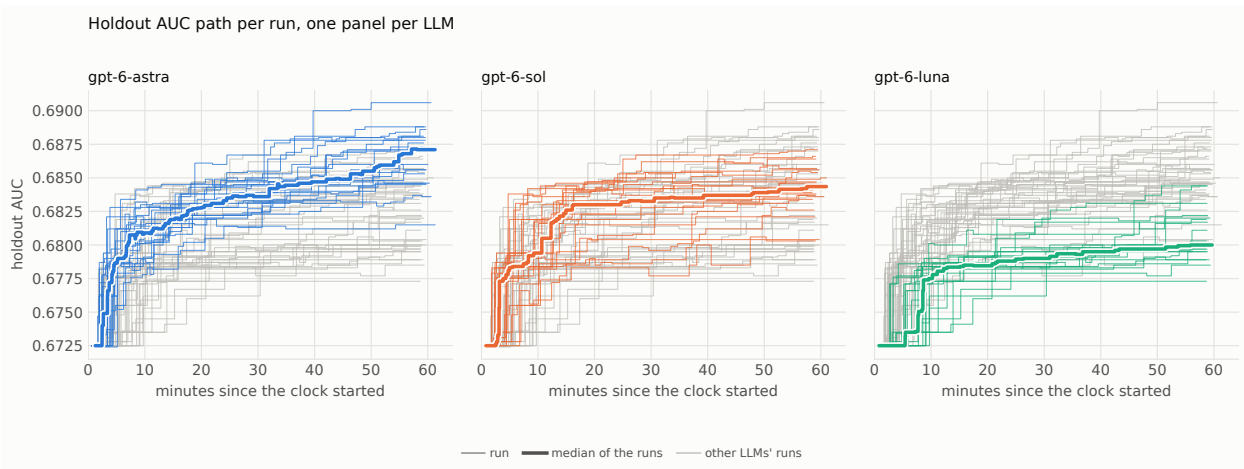


Figure 4: The same paths, one panel per LLM, with the other two LLMs’ runs in gray behind.

Most runs make the same early move. In 19 of Astra’s runs, 19 of Sol’s and 17 of Luna’s, one of the kept changes concerns the depth of the trees (depth 3 or 4 instead of 6, or a limit on the number of leaves), often together with more trees at a lower learning rate, and for Astra and Sol it is typically the first or second change kept, at about three minutes into the hour. By the time it is kept, the holdout AUC is about 0.678, up from the starter’s 0.6725. Simply adding trees at the starter’s learning rate, by contrast, lowers the eval AUC: six Sol runs tried it among their first three

experiments, and all six lost 0.006 to 0.009. A model that fits the training year more closely does worse on the next one.

Table 4: The hour, per LLM. Holdout at t: median over runs of the holdout AUC of the model kept at minute t. Experiments and kept commits: means per run, the baseline counted as a kept commit. Experiment duration: mean time of one experiment, training and evaluation, in seconds. Agent’s share: time outside experiments. Plateau: longest interval without a kept commit (median over runs). Steps down: kept commits whose holdout AUC is below the previous kept commit’s (mean per run).

LLM	holdout at 15 min	30 min	45 min	60 min	experi- ments	kept	experi- ment duration, s	agent’s share of the hour	longest plateau, min	runs with a plateau of 20 min or more	steps down per run
Astra	0.6819	0.6836	0.6849	0.6871	47	19.3	22	72%	10.7	2	2.3
Sol	0.6826	0.6833	0.6837	0.6844	45	13.0	37	54%	14.2	3	1.6
Luna	0.6784	0.6790	0.6797	0.6800	30	8.7	38	68%	19.6	9	0.9

Table 5: The first moves: medians over runs, except the count of runs with a depth change. A kept improvement is a kept commit with a higher eval AUC than the baseline’s; a depth change is one whose description mentions depth, leaves or shallower trees.

LLM	first kept improvement, min	holdout after it	runs with a kept change to tree depth or leaves	its minute	holdout after it
Astra	2.4	0.6766	19	3.4	0.6784
Sol	2.6	0.6775	19	3.2	0.6779
Luna	5.4	0.6760	17	8.6	0.6773

After that common start, the LLMs part ways. The medians of Sol and Astra make their first gain within about three minutes and are neck and neck at the half hour, 0.6833 against 0.6836; after that Sol’s median barely moves, ending at 0.6844, while Astra’s keeps climbing to 0.6871, so most of the gap between the two opens in the second half hour. Luna gets going later, its first gain coming after five minutes rather than two or three, and its median sits between 0.678 and 0.680 from 15 minutes on. Luna also runs fewer experiments in the hour: 30 on average, against 45 for Sol and 47 for Astra, and keeps fewer of them, 9 against 13 and 19, the baseline included. The agents also spend the hour differently. An experiment took 22 seconds on average for Astra and 37 to 38 seconds for Sol and Luna. The rest of the hour, the agent’s own thinking, research and editing, was 72% for Astra, 68% for Luna and 54% for Sol.

Beyond the medians, the paths of single runs diverge. Some runs find a key idea early and climb quickly; others sit on a plateau for a long time. The longest interval without a kept change is 10.7 minutes in the median Astra run, 14.2 in the median Sol run and 19.6 in the median Luna run, and 9 of Luna’s 20 runs went 20 minutes or more without keeping anything, against 3 of Sol’s and 2 of Astra’s. Steps down on the holdout set, kept commits whose holdout AUC is below that of the previous kept commit, happen 2.3 times per Astra run, 1.6 times per Sol run and 0.9 times per Luna run; in 4 Astra, 6 Sol and 3 Luna runs the final model scores below an earlier kept model on the holdout set.

4.5 What the LLMs changed: the calendar

Part of the gap between the LLMs, above all Luna’s gap to the other two, is associated with how they handle the calendar (Table 6, Figure 5; Appendix E describes the audit). The starter model treats the month and the day of the month as categories. Together they identify the exact date, which lets the model pick up patterns of particular days of 2005 that do not carry over to 2006. Astra dropped the day-of-month category in 19 of its 20 runs, often among its first changes, and 15 of its 20 final models use holiday features instead, such as the number of days to the nearest holiday, computed from the row’s own month, day and weekday. Sol dropped it in 13 runs, mostly in favor of the day of the year as a number, and Luna in only 4, its three best runs among them. Within Luna’s runs and within Sol’s, those that dropped it score about 0.003 higher on the holdout set on average than those that kept it (0.0034 and 0.0029). This compares runs rather than isolating one change: the runs that dropped the category made other changes too, and all four Luna runs that dropped it also added a day-of-year feature.

Table 6: How the 20 final models of each LLM handle the calendar, from an audit of their train.py. Day-of-year-like: day, week or fortnight of the year as a number. Means are holdout AUCs of the runs that dropped or kept the day-of-month category (Astra kept it in one run); the difference is that of the means shown.

LLM	dropped the day-of- month category	numeric day-of-year- like feature	holiday features	dropped the month category	mean holdout, dropped	mean holdout, kept	difference
Astra	19	3	15	12	0.6867	0.6836	+0.0031
Sol	13	16	4	8	0.6852	0.6823	+0.0029
Luna	4	5	2	5	0.6834	0.6800	+0.0034

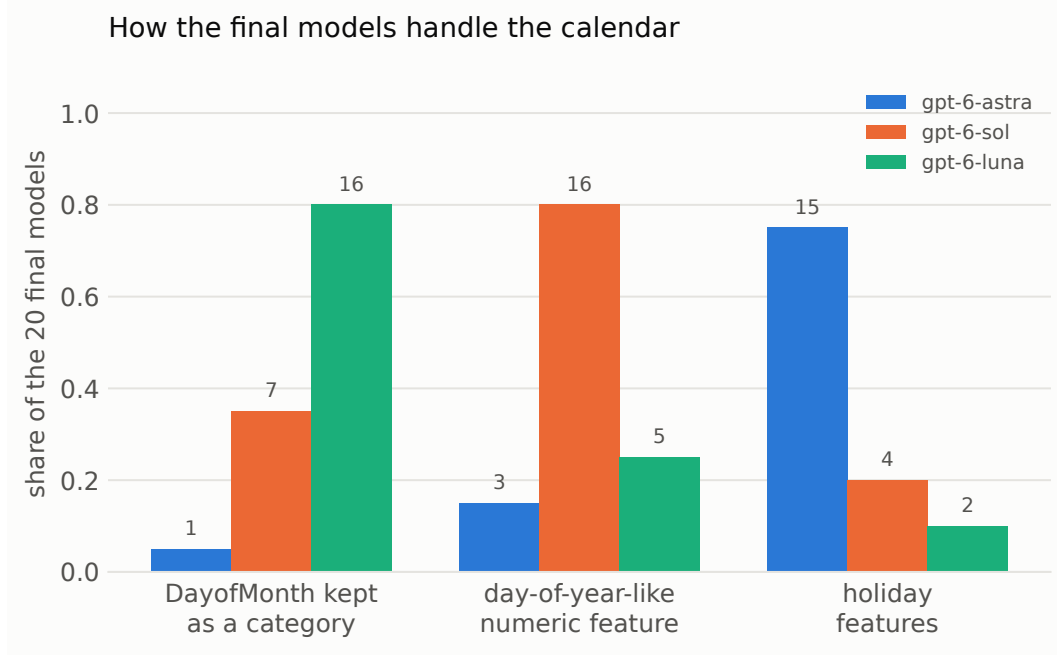


Figure 5: How the final models handle the calendar, by LLM: the share that keeps the day of the month as a category, as the starter does, the share with a numeric day-of-year-like feature, and the share with holiday features.

A good idea alone does not guarantee a good result, though. Astra’s weakest run, at 0.6815, also dropped the day-of-month category and used holiday features; it ended below every Sol run but two. How far a run gets depends partly on luck: which ideas the agent happens to try, in what order, and how well it follows them up.

4.6 Selecting on the evaluation set

The agent selects on the evaluation set, so its final model’s eval AUC can be an optimistic estimate of that model’s quality; the holdout set shows by how much (Table 7, Figure 6). The gap between holdout and eval AUC of the final model is -0.0018 for Luna and Sol, the same as the starter’s own gap, and -0.0028 for Astra, with a standard deviation of 0.0005 to 0.0007 within each LLM. For Luna and Sol the gap is the starter’s own, so selection on the evaluation set left no measurable extra optimism; for Astra, which keeps the most commits and reaches the highest eval AUC, the gap is about 0.001 larger. Eval and holdout AUC rank the runs of an LLM almost identically (rank correlations 0.91 to 0.96), so for choosing between runs the evaluation set is nearly as good as the holdout set.

Table 7: Eval and holdout AUC of the final models and their gap (holdout minus eval). The starter’s gap is -0.0018.

LLM	mean eval AUC	mean holdout AUC	mean gap	sd	min	max	rank correlation of eval and holdout AUC
Astra	0.6893	0.6866	-0.0028	0.0005	-0.0037	-0.0019	0.94
Sol	0.6860	0.6842	-0.0018	0.0007	-0.0036	-0.0006	0.91
Luna	0.6825	0.6806	-0.0018	0.0006	-0.0032	-0.0006	0.96

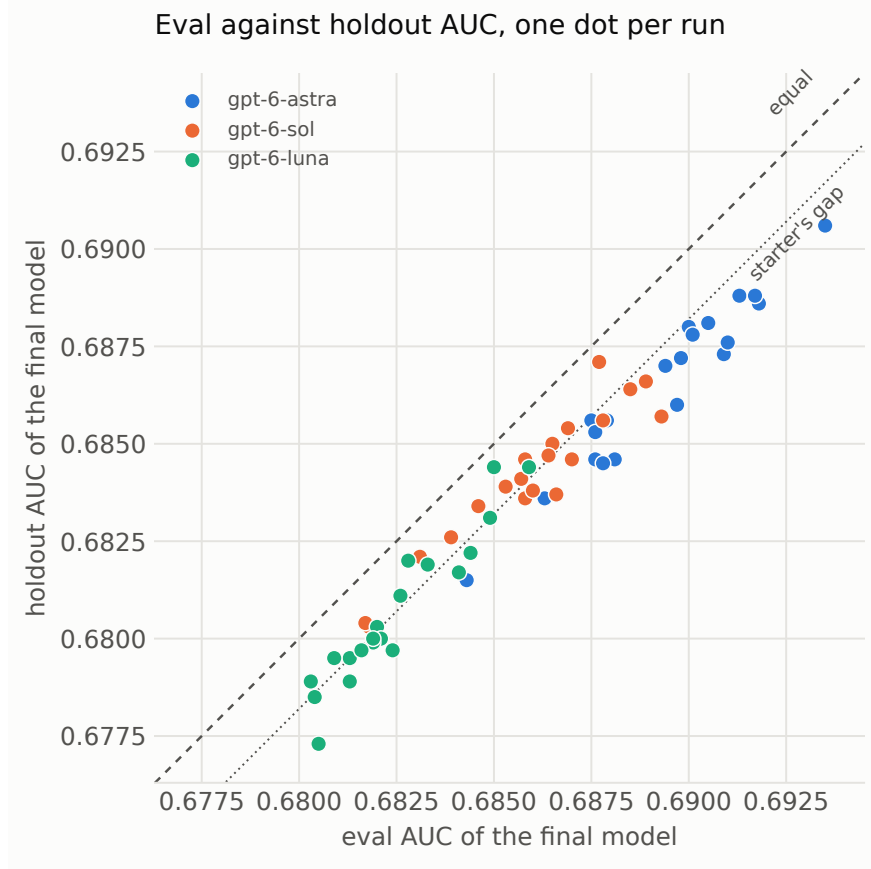


Figure 6: Eval AUC against holdout AUC of each run’s final model. The dashed line is equality; the dotted line is the starter model’s gap.

4.7 Several runs, pick the best

If the goal is not to compare LLMs but to get a good model, the variation between runs can be turned to advantage: run the agent several times and choose the run with the best eval AUC. Table 8 and Figure 7 give the holdout AUC of the chosen run, computed exactly over the 20 observed runs of each LLM. As k grows, the chosen run is more and more often one of the few runs with the highest eval AUC, so the curves flatten. For Sol those runs are not its best on the holdout set: its run with the best eval AUC scores 0.6857, its best run 0.6871. Three runs instead of one raise the median by 0.0010 for Astra, 0.0012 for Sol and 0.0020 for Luna, and they raise the floor more than the median: the 5th percentile of the chosen run’s holdout AUC moves from 0.6815, 0.6803 and 0.6773 (one run) to 0.6856, 0.6837 and 0.6797 (three runs). With three runs, choosing on the evaluation set costs nothing: an oracle choosing on the holdout set itself would choose runs with the same median holdout AUC. With five or ten runs the oracle would do better for Sol, by 0.0007 and 0.0009, because Sol’s eval and holdout AUC disagree about which of its best runs is best. For Astra and Sol the median hardly moves beyond five runs; Luna, whose two best runs stand well above the rest, keeps gaining up to ten. This analysis was defined after the runs, and it is only one task; its point is the shape of the curve, not the exact numbers.

Table 8: Running the agent k times and choosing the run with the best eval AUC: median holdout AUC of the chosen run, computed exactly over the 20 observed runs with draws with replacement (for $k = 1$, the median of Table 2). The last two columns give, for $k = 3$, the 5th percentile of the chosen run’s holdout AUC, from the same exact distribution, and the median if the choice were made on the holdout set itself.

LLM	$k = 1$	$k = 2$	$k = 3$	$k = 5$	$k = 10$	$k = 3$, 5th percentile	$k = 3$, chosen on holdout
Astra	0.6871	0.6878	0.6881	0.6886	0.6888	0.6856	0.6881
Sol	0.6844	0.6854	0.6856	0.6857	0.6857	0.6837	0.6856
Luna	0.6800	0.6818	0.6820	0.6831	0.6844	0.6797	0.6820

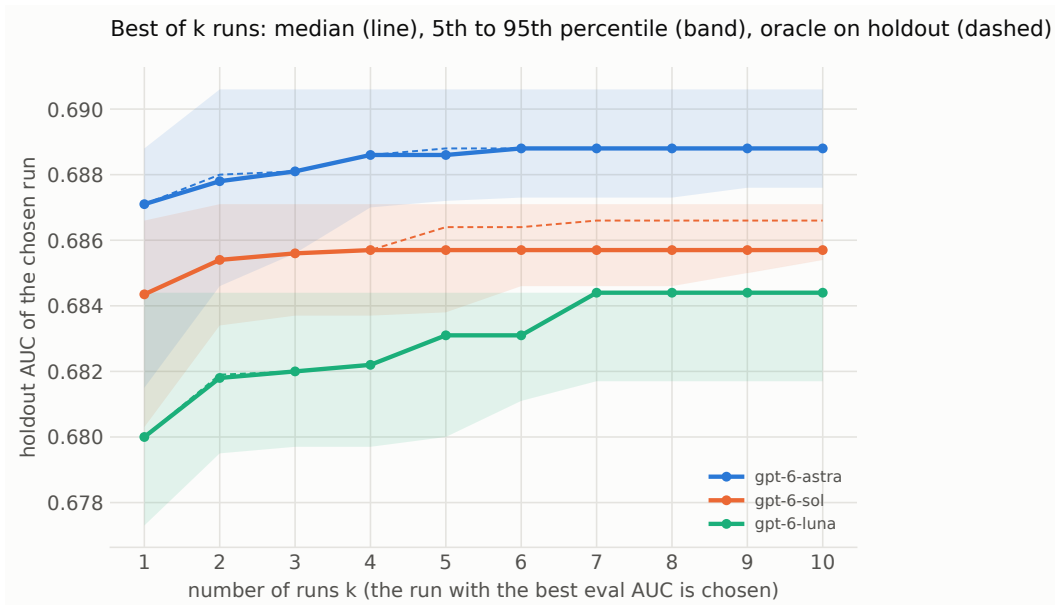


Figure 7: Running the agent k times and choosing the run with the best eval AUC: the median holdout AUC of the chosen run (line), its 5th to 95th percentile (band), and the median if the choice were made on the holdout set (dashed).

Selecting the best of several runs on a set and then reporting that set’s score overstates the chosen model’s quality, by an amount that grows with the number of candidates (Cawley and Talbot, 2010; Dwork et al., 2015b). Here the holdout set was not used for selection, so its scores are unbiased for the final models. A team that selects among runs on its evaluation set should therefore confirm the chosen model on data that neither the agent nor the selection has touched, as the holdout set does in this study; a team that selects on its holdout set needs a third set for an unbiased estimate.

4.8 Validity of the runs

No run was excluded, no violation of the data rules was found, and the leak checks found no content of the holdout set, the evaluation set or the human-only scripts in any session log. The ten flags raised by the integrity checks, the leak check included, were all false alarms on review, and the ten protocol caveats are kept ties and one run’s retry waits (Section 3.6, Appendix B). Leaving out the ten caveat runs, or the three runs that read documentation about the evaluation year, moves the three means by at most 0.0002 and the Astra-versus-Sol win probability to 82% and

79% respectively (Appendix B). Every run’s agent stopped the clock itself; the clock ran for 58 to 61 minutes (six runs ended their wrap-up after the budget had run out, with no experiment started after it). Five runs met service errors (“model at capacity”) and were resumed by the driver; context compactions, the agent summarizing its own history when its context filled, happened in 25 runs, mostly late in the hour, and disturbed no run’s protocol. Table 14 in Appendix F gives the operational details and Table 16 in Appendix H the token usage.

5 Discussion

5.1 Comparing agents, LLMs and prompts

A single run of an agent is one draw from a distribution that, on this task, is as wide as the differences between LLMs of clearly different strength. Anyone comparing agents, LLMs, prompts, effort settings or any other part of a setup on the basis of one run each is therefore comparing two draws, and the head-to-head probabilities of Section 4.3 say how often that goes wrong: one time in five for the two closest LLMs here, whose means differ by 3.7 standard errors. The remedy is the one the older literature on seeds arrived at long ago: run repeatedly, report the distribution, and choose the number of runs from the difference one wants to detect (Colas et al., 2018; Miller, 2024). On this task, with a run-to-run standard deviation of about 0.002, the approximation of Appendix D puts the number of runs per arm at about twelve for the gap between Astra and Sol and at two for the gap between Astra and Luna; these are post-hoc illustrations, since the gaps were observed rather than fixed in advance, but they give the scale. Twenty runs per LLM were enough to rank the three LLMs with confidence; they still leave the head-to-head probability for the closest pair anywhere between 66% and 93%.

5.2 Getting a good model

If the goal is not a comparison but a model, the same variation is an opportunity. Running the agent three times instead of once and choosing the run with the best eval AUC raised the median holdout AUC by 0.001 to 0.002 and raised the floor by more. With three runs, choosing on the evaluation set cost nothing against choosing on the holdout set, because the two rank the runs almost identically; with five or more, it cost Sol up to 0.0009. A team that adopts this policy selects on its evaluation set and should then confirm the chosen model on data that neither the agent nor the selection has touched, since the eval AUC of a model chosen for its eval AUC can be optimistic twice over, once through the agent’s keep rule, which here added about 0.001 for Astra and nothing measurable for Luna and Sol, and once through the selection among runs (Cawley and Talbot, 2010). The holdout set plays that role in this study; a team that selects on its holdout set needs a third set. Three to five runs capture most of the gain for Astra and Sol; beyond that the chosen run is more and more often one of the few runs with the highest eval AUC, and more runs add little. Luna, with two outstanding runs among its 20, kept gaining up to ten.

5.3 Why the LLMs differ, and why runs of one LLM differ

The stronger LLMs run more experiments in the hour (45 to 47 against Luna’s 30), make their first gain earlier and keep more commits. Astra also keeps climbing longer: its advantage over Sol opened mostly in the second half hour, when Sol’s median had flattened. Part of the difference is associated with a single modeling decision. The starter treats the day of the month as a category, which, together with the month, identifies the date and lets the model learn the particular days of

2005. Astra dropped that category in 19 of 20 runs and mostly replaced it with holiday features; Sol dropped it in 13, mostly for the day of the year as a number; Luna dropped it in 4, its three best runs among them. Within Luna’s runs and within Sol’s, the runs that dropped it scored about 0.003 higher on the holdout set, half the gap between Luna’s and Astra’s means, though they differ in other changes too. All runs were told about the year shift; whether an LLM acts on it by dropping the category, and how early, depends partly on what it knows about tabular modeling under such a shift and partly on chance: which ideas it happens to try first, in what order, and how well it follows them up. Astra’s weakest run found the idea and still ended below all but two of Sol’s runs. The spread within an LLM is the trace of that chance.

5.4 What the design choices mean for the results

Several choices shape what we measured. The budget is an hour of wall-clock time, not a number of experiments, so an LLM that thinks faster, keeps its turns short or writes code that runs faster gets more experiments: Astra ran the most while spending the largest share of the hour on its own work, because its experiments were faster. That is part of what is being compared, as it is for a user paying for an hour of the agent’s time. Within an LLM, runs with more experiments tend to score somewhat higher (Figure 8, exploratory), so a budget in experiments might narrow the gaps. The effort setting is the one named “max” for all three LLMs, not each LLM’s ceiling: Sol and Astra have one level above it in Codex’s catalog. Reasoning models generally do better when allowed to think longer (OpenAI, 2024), so at “ultra” the results of Sol and Astra, and the gaps between the LLMs, could differ from those reported here. The evaluation set is scored one row at a time, which rules out a class of features (counts and group statistics of the rows being scored) that would otherwise be available and would make the eval AUC meaningless; the agents were told why. The holdout set comes from the same year as the evaluation set, so the gaps we report are those of the tuning year; a later year would be a stricter test and is future work. The web research the instructions require is a source of variation of its own, and three runs used what they read about the evaluation year, two about carrier reporting and two about holiday travel seasons, one run doing both; we report them and keep them, since they read documentation, not flight data.

5.5 Limitations

One agent, three LLMs from one vendor, one task and dataset, one hour, one effort setting, and 20 runs per LLM. The sizes we report are those of this setup; other tasks, longer budgets or other agents will have other spreads, and the orchestrator exists so that they can be measured rather than assumed. The LLMs are hosted services; the runs were made over three days through those services, and whatever changed in them meanwhile is part of the variation we report, as it would be for any user. The feature audit was scripted, and every file was then read in full by AI agents under written rules (Appendix E); it classifies code, and borderline cases are judgment calls. The automatic checks look for the shortcuts we anticipated; a shortcut of a kind no check looks for would pass, though the leak check’s content search and the row-by-row scoring close the obvious ones. The review of each run was itself done by an LLM agent with the authors reviewing its reviews; the integrity verdicts are documented per run and can be checked against the archived logs. Finally, the best-of-k and the runs-per-arm analyses were defined after the runs, on the same 60 runs, and are presented as illustrations rather than as estimates.

5.6 Future work

The orchestrator makes the obvious extensions cheap: other agents, other LLMs and effort settings, other prompts, longer budgets, more runs for close pairs, and other tasks and datasets. Scoring the delivered models on a later year would show how much of the gain survives beyond the tuning year. A budget in experiments rather than time would separate the LLMs’ speed from their judgment. And the session logs, which are archived with every run, hold material we have barely used: what each LLM searched for, what it read, and how its hypotheses followed from it.

6 Conclusion

On this task, all 60 of our runs beat the starter model on the holdout set, by 0.008 to 0.014 AUC on average depending on the LLM. But the size of the improvement varies a lot from run to run, even with identical setup and instructions: by 0.007 to 0.009 AUC from an LLM’s worst run to its best, more than the difference between the best and the worst LLM’s averages. This has two practical consequences. First, to compare agents, LLMs, prompts or any other part of a setup, one run is not enough. Repeated runs are needed, and the comparison should look at the distribution of the results, not at a single number; here, one run of each would have ranked the two closest LLMs wrongly one time in five. Second, if the goal is a good model, it pays to run the agent several times and choose the run with the best score on the evaluation set, which with three runs chose as well as the holdout set would have, and then to confirm the chosen model on data that neither the agent nor the selection has touched. Both the single-run building block and the multi-run orchestrator are open source, so others can test their own agents, LLMs and setups, ideally with more than one run each.

References

- Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*, 2021. URL <https://arxiv.org/abs/2108.13264>.
- Eduardo Ariño de la Rubia and Szilard Pafka. Identical runs, different results: Benchmarking AI coding agents on open-weight models, September 2026. URL <https://arxiv.org/abs/2609.33812>.
- Berk Atil, Sarp Aykent, Alexa Chittams, Lisheng Fu, Rebecca J. Passonneau, Evan Radcliffe, Guru Rajan Rajagopal, Adam Sloan, Tomasz Tudrej, Ferhan Ture, Zhe Wu, Lixinyu Xu, and Breck Baldwin. Non-determinism of “deterministic” LLM settings, 2025. URL <https://arxiv.org/abs/2408.04667>.
- Bjarni Haukur Bjarnason, André Silva, and Martin Monperrus. On randomness in agentic evals, 2026. URL <https://arxiv.org/abs/2602.07150>.
- Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, Brennan Nichyporuk, Justin Szeto, Naz Sepah, Edward Raff, Kanika Madan, Vikram Voleti, Samira Ebrahimi Kahou, Vincent Michalski, Dmitriy Serdyuk, Tal Arbel, Chris Pal, Gaël Varoquaux, and Pascal Vincent. Accounting for variance in machine learning benchmarks. In A. Smola, A. Dimakis, and I. Stoica, editors, *Proceedings of Machine Learning and Systems*, volume 3, pages 747–769, 2021. doi: 10.48550/arXiv.2103.03098. URL https://proceedings.mlsys.org/paper_files/paper/2021/hash/0184b0cd3cfb185989f858a1d9f5c1eb-Abstract.html.

- Sam Bowyer, Laurence Aitchison, and Desi R. Ivanova. Position: Don’t use the CLT in LLM evals with fewer than a few hundred datapoints. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 81143–81184. PMLR, 2025. URL <https://proceedings.mlr.press/v267/bowyer25a.html>.
- Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling, 2024. URL <https://arxiv.org/abs/2407.21787>.
- Hao-Run Cai and Han-Jia Ye. Understanding the limits of deep tabular methods with temporal shift. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025. URL <https://arxiv.org/abs/2502.20260>.
- Gavin C. Cawley and Nicola L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11(70):2079–2107, 2010. URL <https://www.jmlr.org/papers/v11/cawley10a.html>.
- Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Mądry. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *International Conference on Learning Representations (ICLR)*, pages 50466–50494, 2025. doi: 10.48550/arXiv.2410.07095. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/7e3767db483c942b883eb4f8cfb74e31-Abstract-Conference.html.
- Hardy Chen, Nancy Lau, Haoqin Tu, Shuo Yan, Xiangyan Liu, Zijun Wang, Juncheng Wu, Michael Qizhe Shieh, Alvaro A. Cardenas, Cihang Xie, and Yuyin Zhou. Chasing the public score: User pressure and evaluation exploitation in coding agent workflows, 2026. URL <https://arxiv.org/abs/2604.20200>.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, et al. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>.
- Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD ’16, pages 785–794, San Francisco, California, USA, 2016. ACM. doi: 10.1145/2939672.2939785. URL <https://doi.org/10.1145/2939672.2939785>.
- Cédric Colas, Olivier Sigaud, and Pierre-Yves Oudeyer. How many random seeds? Statistical power analysis in deep reinforcement learning experiments, 2018. URL <https://arxiv.org/abs/1806.08295>.
- Data Expo 2009. Data Expo 2009: Airline on time data, 2008. URL <https://doi.org/10.7910/DVN/HG7NV7>.
- Elizabeth R. DeLong, David M. DeLong, and Daniel L. Clarke-Pearson. Comparing the areas under two or more correlated receiver operating characteristic curves: A nonparametric approach. *Biometrics*, 44(3):837–845, 1988. doi: 10.2307/2531595. URL <https://doi.org/10.2307/2531595>.
- Jesse Dodge, Gabriel Ilharco, Roy Schwartz, Ali Farhadi, Hannaneh Hajishirzi, and Noah Smith. Fine-tuning pretrained language models: Weight initializations, data orders, and early stopping, 2020. URL <https://arxiv.org/abs/2002.06305>.
- Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. Generalization in adaptive data analysis and holdout reuse. In *Advances in Neural*

- Information Processing Systems*, volume 28, 2015a. doi: 10.48550/arXiv.1506.02629. URL https://papers.nips.cc/paper_files/paper/2015/hash/bad5f33780c42f2588878a9d07405083-Abstract.html.
- Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248):636–638, 2015b. doi: 10.1126/science.aaa9375. URL <https://doi.org/10.1126/science.aaa9375>.
- B. Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, January 1979. doi: 10.1214/aos/1176344552. URL <https://doi.org/10.1214/aos/1176344552>.
- Run-Ze Fan, Zihao Zhang, Simin Ma, Yebowen Hu, Shouju Wang, Kaiqiang Song, Fei Liu, Hamed Zamani, and Xiaoyang Wang. An empirical study of harness design for coding agents, 2026. URL <https://arxiv.org/abs/2609.20804>.
- Fabio Ferreira, Lucca Wobbe, Arjun Krishnakumar, Frank Hutter, and Arber Zela. Can LLMs beat classical hyperparameter optimization algorithms? A study on autoresearch, 2026. URL <https://arxiv.org/abs/2603.24647>.
- Josh Gardner, Zoran Popović, and Ludwig Schmidt. Benchmarking distribution shift in tabular data with TableShift. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2023. URL <https://arxiv.org/abs/2312.07577>.
- Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, volume 35, 2022. URL <https://arxiv.org/abs/2207.08815>.
- Antoine Grosnit, Alexandre Maraval, Refinath S N, Zichao Zhao, James Doran, Giuseppe Paolo, Albert Thomas, Jonas Gonzalez, Abhineet Kumar, Khyati Khandelwal, Abdelhakim Benezhab, Hamza Cherkaoui, Youssef Attia El-Hili, Kun Shao, Jianye Hao, Jun Yao, Balázs Kégl, Haitham Bou-Ammar, and Jun Wang. Kolb-based experiential learning for generalist agents with human-level Kaggle data science performance, 2024. URL <https://arxiv.org/abs/2411.03562>.
- Siyuan Guo, Cheng Deng, Ying Wen, Hechang Chen, Yi Chang, and Jun Wang. DS-Agent: Automated data science by empowering large language models with case-based reasoning. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. URL <https://arxiv.org/abs/2402.17453>.
- James A. Hanley and Barbara J. McNeil. The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology*, 143(1):29–36, 1982. doi: 10.1148/radiology.143.1.7063747. URL <https://doi.org/10.1148/radiology.143.1.7063747>.
- Mohsen Hariri, Amirhossein Samandar, Michael Hinczewski, and Vipin Chaudhary. Don’t Pass@k: A Bayesian framework for large language model evaluation. In *International Conference on Learning Representations (ICLR)*, 2026. URL <https://arxiv.org/abs/2510.04265>.
- Horace He and Thinking Machines Lab. Defeating nondeterminism in LLM inference. Thinking Machines Lab: Connectionism (blog), September 2025. URL <https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/>.
- Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, pages 3207–3214, 2018. doi: 10.1609/aaai.v32i1.11694. URL <https://ojs.aaai.org/index.php/AAAI/article/view/11694>.

- Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. MAgentBench: Evaluating language agents on machine learning experimentation. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 20271–20309. PMLR, 2024. URL <https://proceedings.mlr.press/v235/huang24y.html>.
- Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. AIDE: AI-driven exploration in the space of code, 2025. URL <https://arxiv.org/abs/2502.13138>.
- Liqiang Jing, Zhehui Huang, Xiaoyang Wang, Wenlin Yao, Wenhao Yu, Kaixin Ma, Hongming Zhang, Xinya Du, and Dong Yu. DS Bench: How far are data science agents from becoming data science experts? In *International Conference on Learning Representations (ICLR)*, pages 32597–32649, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/50e9ad960ae78b741a6b4fea533f2eaf-Abstract-Conference.html.
- Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. AI agents that matter, June 2025. URL <https://openreview.net/forum?id=Zy4uFzMviZ>.
- Andrej Karpathy. autoresearch: AI agents running research on single-GPU nanochat training automatically. GitHub repository, March 2026. URL <https://github.com/karpathy/autoresearch>.
- Sydney Lewis. Same model, different harness: Different coding-agent results, 2026. URL <https://arxiv.org/abs/2608.26218>.
- Ziming Li, Qianbo Zang, David Ma, Jiawei Guo, Tuney Zheng, Minghao Liu, Xinyao Niu, Yue Wang, Jian Yang, Jiaheng Liu, Wanjun Zhong, Wangchunshu Zhou, Wenhao Huang, and Ge Zhang. AutoKaggle: A multi-agent framework for autonomous data science competitions, 2024. URL <https://arxiv.org/abs/2410.20424>.
- Siyi Liu, Chen Gao, and Yong Li. Large language model agent for hyper-parameter optimization, 2024a. URL <https://arxiv.org/abs/2402.01881>.
- Tennison Liu, Nicolás Astorga, Nabeel Seedat, and Mihaela van der Schaar. Large language models to enhance Bayesian optimization. In *International Conference on Learning Representations (ICLR)*, 2024b. URL <https://arxiv.org/abs/2402.03921>.
- Lovish Madaan, Aaditya K. Singh, Rylan Schaeffer, Andrew Poulton, Sanmi Koyejo, Pontus Stenertorp, Sharan Narang, and Dieuwke Hupkes. Quantifying variance in evaluation benchmarks, 2024. URL <https://arxiv.org/abs/2406.10229>.
- H. B. Mann and D. R. Whitney. On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics*, 18(1):50–60, March 1947. doi: 10.1214/aoms/1177730491. URL <https://doi.org/10.1214/aoms/1177730491>.
- Kenneth O. McGraw and S. P. Wong. A common language effect size statistic. *Psychological Bulletin*, 111(2):361–365, 1992. doi: 10.1037/0033-2909.111.2.361. URL <https://doi.org/10.1037/0033-2909.111.2.361>.
- Aman Mehta. When agents disagree with themselves: Behavioral consistency as an uncertainty signal for LLM agents, 2026. URL <https://arxiv.org/abs/2602.11619>.
- METR. Details about METR’s preliminary evaluation of OpenAI’s o3 and o4-mini, April 2025. URL <https://metr.org/evaluations/openai-o3-report/>.

- Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations, 2024. URL <https://arxiv.org/abs/2411.00640>.
- Josias Moukpe, Priyanka Aryal, and Matthew Kenney. DeltaML-Bench: Evaluating machine learning agents on real-world research repositories, 2026. URL <https://arxiv.org/abs/2608.19653>.
- Jaehyun Nam, Jinsung Yoon, Jiefeng Chen, Jinwoo Shin, Sercan Ö. Arik, and Tomas Pfister. MLE-STAR: Machine learning engineering agent via search and targeted refinement, 2025. URL <https://arxiv.org/abs/2506.15692>.
- OpenAI. Learning to reason with LLMs. Research announcement of OpenAI o1, September 2024. URL <https://openai.com/index/learning-to-reason-with-llms/>.
- OpenAI. Codex CLI, version 0.160.0. Software release rust-v0.160.0, GitHub, October 2026a. URL <https://github.com/openai/codex/releases/tag/rust-v0.160.0>.
- OpenAI. Codex configuration reference. Developer documentation, config.toml reference, 2026b. URL <https://developers.openai.com/codex/config-reference>.
- OpenAI. GPT-6 model family: GPT-6 Astra, GPT-6 Sol and GPT-6 Luna. Model documentation, OpenAI API reference, 2026c. URL <https://developers.openai.com/api/docs/models/gpt-6-astra>.
- Shuyin Ouyang, Jie M. Zhang, Mark Harman, and Meng Wang. An empirical study of the non-determinism of ChatGPT in code generation. *ACM Transactions on Software Engineering and Methodology*, 34(2):1–28, February 2025. doi: 10.1145/3697010. URL <https://doi.org/10.1145/3697010>.
- Szilard Pafka. benchm-ml: A minimal benchmark for scalability, speed and accuracy of commonly used open source implementations of the top machine learning algorithms for binary classification. GitHub repository, March 2015. URL <https://github.com/szilard/benchm-ml>.
- Szilard Pafka. xgboost-autoresearch-minimal3: a minimal, self-contained setup for one autonomous XGBoost tuning run. GitHub repository, 2026a. URL <https://github.com/szilard/xgboost-autoresearch-minimal3>.
- Szilard Pafka. xgboost-autoresearch-minimal3-runs: orchestrator and archived runs. GitHub repository, 2026b. URL <https://github.com/szilard/xgboost-autoresearch-minimal3-runs>.
- Szilard Pafka and Eduardo Ariño de la Rubia. Automating data science: Optimizing XGBoost machine learning models with AI agents, April 2026a. URL <https://szilard.github.io/xgboost-autoresearch/>.
- Szilard Pafka and Eduardo Ariño de la Rubia. One run is not enough: How much AI agent results vary in data science, an XGBoost optimization study, October 2026b. URL <https://szilard.github.io/xgboost-autoresearch3/>.
- Melissa Z. Pan, Shuo Yang, Negar Arabzadeh, Wei-Lin Chiang, Ion Stoica, and Matei Zaharia. HarnessTax: How much does the harness matter for coding agents?, 2026. URL <https://harnesstax.github.io/>.
- Stephan Rabanser, Sayash Kapoor, Peter Kirgis, Kangheng Liu, Saiteja Utpala, and Arvind Narayanan. Towards a science of AI agent reliability. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 102769–102820. PMLR, 2026. URL <https://proceedings.mlr.press/v306/rabanser26a.html>.

- Mizanur Rahman, Mohammed Saidul Islam, Ridwan Mahbub, Md Tahmid Rahman Laskar, Shafiq Joty, and Enamul Hoque Prince. DSAgentBench: Can agents automate end-to-end data-science workflows in real computer environments?, 2026. URL <https://arxiv.org/abs/2608.10366>.
- Carson Rodrigues, Oysturn Vas, Isaiah Abner DCosta, and Nithish Kumar Prabhakaran. When is an LLM worth it for hyperparameter optimization? A budget-matched study on tabular data finds the warm-start is a default configuration, not the model, 2026. URL <https://arxiv.org/abs/2606.21641>.
- Rebecca Roelofs, Vaishaal Shankar, Benjamin Recht, Sara Fridovich-Keil, Moritz Hardt, John Miller, and Ludwig Schmidt. A meta-analysis of overfitting in machine learning. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d’Alché Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/ee39e503b6bedf0c98c388b7e8589aca-Abstract.html>.
- Ivan Rubachev, Nikolay Kartashev, Yury Gorishniy, and Artem Babenko. TabReD: Analyzing pitfalls and filling the gaps in tabular deep learning benchmarks. In *International Conference on Learning Representations (ICLR)*, pages 35166–35202, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/571799482291411607c54984153190b0-Abstract-Conference.html.
- Ravid Shwartz-Ziv and Amitai Armon. Tabular data: Deep learning is not all you need. *Information Fusion*, 81:84–90, May 2022. doi: 10.1016/j.inffus.2021.11.011. URL <https://doi.org/10.1016/j.inffus.2021.11.011>.
- András Vargha and Harold D. Delaney. A critique and improvement of the CL common language effect size statistics of McGraw and Wong. *Journal of Educational and Behavioral Statistics*, 25(2):101–132, 2000. doi: 10.3102/10769986025002101.
- Sida Wang. Measuring all the noises of LLM evals, 2025. URL <https://arxiv.org/abs/2512.21326>.
- B. L. Welch. The generalization of ‘Student’s’ problem when several different population variances are involved. *Biometrika*, 34(1–2):28–35, January 1947. doi: 10.1093/biomet/34.1-2.28. URL <https://doi.org/10.1093/biomet/34.1-2.28>.
- Hjalmar Wijk, Tao Roa Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, Lawrence Chan, Michael Chen, Joshua M Clymer, Jai Dhyani, Elena Elicheva, Katharyn Garcia, Brian Goodrich, Nikola Jurkovic, Megan Kinniment, Aron Lajko, Seraphina Nix, Lucas Jun Koba Sato, William Saunders, Maksym Taran, Ben West, and Elizabeth Barnes. RE-bench: Evaluating frontier AI R&D capabilities of language model agents against human experts. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 66772–66832. PMLR, 2025. URL <https://proceedings.mlr.press/v267/wijk25a.html>.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations (ICLR)*, pages 9965–10017, 2025. doi: 10.48550/arXiv.2406.12045. URL <https://arxiv.org/abs/2406.12045>.
- Michael R. Zhang, Nishkrit Desai, Juhan Bae, Jonathan Lorraine, and Jimmy Ba. Using large language models for hyperparameter optimization. In *NeurIPS 2023 Workshop on Foundation Models for Decision Making*, 2023. URL <https://arxiv.org/abs/2312.04528>.
- Bingchen Zhao, Dhruv Srikanth, Yuxiang Wu, and Zhengyao Jiang. SpecBench: Measuring reward hacking in long-horizon coding agents, 2026. URL <https://arxiv.org/abs/2605.21384>.

Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, Andy Zhang, Shu Liu, Sasha Cui, Sayash Kapoor, Shayne Longpre, Kevin Meng, Rebecca Weiss, Fazl Barez, Rahul Gupta, Jwala Dhamala, Jacob Merizian, Mario Giulianelli, Harry Coppock, Cozmin Ududec, Jasjeet Sekhon, Jacob Steinhardt, Antony Kellermann, Sarah Schwettmann, Matei Zaharia, Ion Stoica, Percy Liang, and Daniel Kang. Establishing best practices for building rigorous agentic benchmarks. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2025. URL <https://arxiv.org/abs/2507.02825>.

A The 60 runs

Run names give the LLM, the group of 20 runs and the run number: astra6_n20-7 is the seventh of the 20 runs of gpt-6-astra.

Table 9: The 60 runs. Experiments: rows of results.tsv, the baseline included. Kept: kept commits, the baseline included. Eval and holdout AUC: of the final model. Clock: from start to stop. Agent’s share: time outside experiments.

run	experiments	kept	eval AUC	holdout AUC	gap	clock	agent’s share	caveat
astra6_n20-1	47	19	0.6900	0.6880	-0.0020	59m34s	85%	
astra6_n20-2	42	16	0.6863	0.6836	-0.0027	60m34s	51%	
astra6_n20-3	50	24	0.6918	0.6886	-0.0032	59m38s	73%	
astra6_n20-4	36	13	0.6843	0.6815	-0.0028	61m18s	68%	
astra6_n20-5	47	16	0.6876	0.6846	-0.0030	60m16s	75%	keep_rule
astra6_n20-6	41	12	0.6881	0.6846	-0.0035	59m39s	79%	
astra6_n20-7	43	23	0.6905	0.6881	-0.0024	59m13s	85%	
astra6_n20-8	50	21	0.6879	0.6856	-0.0023	59m40s	78%	keep_rule
astra6_n20-9	36	18	0.6913	0.6888	-0.0025	59m14s	68%	
astra6_n20-10	45	20	0.6878	0.6845	-0.0033	59m41s	70%	
astra6_n20-11	52	20	0.6897	0.6860	-0.0037	59m59s	84%	
astra6_n20-12	51	24	0.6909	0.6873	-0.0036	59m24s	68%	keep_rule
astra6_n20-13	59	27	0.6935	0.6906	-0.0029	60m32s	64%	
astra6_n20-14	62	26	0.6917	0.6888	-0.0029	59m37s	70%	
astra6_n20-15	54	12	0.6875	0.6856	-0.0019	59m32s	66%	
astra6_n20-16	49	21	0.6894	0.6870	-0.0024	59m30s	68%	
astra6_n20-17	46	17	0.6898	0.6872	-0.0026	58m42s	62%	
astra6_n20-18	44	16	0.6901	0.6878	-0.0023	59m32s	66%	
astra6_n20-19	49	23	0.6910	0.6876	-0.0034	59m14s	80%	
astra6_n20-20	40	17	0.6876	0.6853	-0.0023	59m25s	73%	
sol6_n20-1	34	9	0.6865	0.6850	-0.0015	60m59s	68%	
sol6_n20-2	34	9	0.6858	0.6836	-0.0022	60m28s	64%	
sol6_n20-3	35	8	0.6857	0.6841	-0.0016	58m34s	60%	
sol6_n20-4	29	5	0.6831	0.6821	-0.0010	58m55s	71%	
sol6_n20-5	47	22	0.6839	0.6826	-0.0013	58m46s	40%	
sol6_n20-6	43	15	0.6818	0.6803	-0.0015	59m02s	48%	
sol6_n20-7	58	19	0.6866	0.6837	-0.0029	59m01s	43%	
sol6_n20-8	48	19	0.6877	0.6871	-0.0006	59m22s	45%	keep_rule
sol6_n20-9	51	12	0.6846	0.6834	-0.0012	58m28s	41%	
sol6_n20-10	51	13	0.6885	0.6864	-0.0021	59m06s	49%	
sol6_n20-11	55	16	0.6893	0.6857	-0.0036	59m15s	47%	
sol6_n20-12	70	17	0.6870	0.6846	-0.0024	58m52s	43%	
sol6_n20-13	51	16	0.6869	0.6854	-0.0015	59m23s	55%	
sol6_n20-14	43	11	0.6858	0.6846	-0.0012	59m12s	57%	
sol6_n20-15	39	12	0.6878	0.6856	-0.0022	59m39s	64%	
sol6_n20-16	31	5	0.6817	0.6804	-0.0013	59m42s	70%	turn_retries
sol6_n20-17	35	13	0.6853	0.6839	-0.0014	58m26s	55%	
sol6_n20-18	60	10	0.6864	0.6847	-0.0017	59m08s	40%	
sol6_n20-19	58	22	0.6889	0.6866	-0.0023	59m02s	41%	keep_rule
sol6_n20-20	30	7	0.6860	0.6838	-0.0022	59m09s	71%	
luna6_n20-1	36	13	0.6844	0.6822	-0.0022	58m51s	59%	
luna6_n20-2	25	5	0.6813	0.6789	-0.0024	59m07s	75%	
luna6_n20-3	29	11	0.6820	0.6803	-0.0017	58m37s	68%	

run	experiments	kept	eval AUC	holdout AUC	gap	clock	agent's share	caveat
luna6_n20-4	37	10	0.6824	0.6797	-0.0027	58m44s	62%	
luna6_n20-5	25	8	0.6803	0.6789	-0.0014	58m31s	73%	
luna6_n20-6	30	9	0.6821	0.6800	-0.0021	58m54s	59%	keep_rule
luna6_n20-7	25	8	0.6813	0.6795	-0.0018	58m32s	79%	
luna6_n20-8	28	7	0.6809	0.6795	-0.0014	58m51s	73%	keep_rule
luna6_n20-9	24	9	0.6859	0.6844	-0.0015	58m35s	70%	
luna6_n20-10	32	11	0.6828	0.6820	-0.0008	59m15s	69%	
luna6_n20-11	34	3	0.6805	0.6773	-0.0032	58m40s	68%	
luna6_n20-12	26	10	0.6819	0.6799	-0.0020	58m47s	73%	
luna6_n20-13	30	8	0.6816	0.6797	-0.0019	58m36s	70%	
luna6_n20-14	35	6	0.6819	0.6800	-0.0019	59m42s	66%	
luna6_n20-15	25	8	0.6804	0.6785	-0.0019	59m20s	69%	keep_rule
luna6_n20-16	26	11	0.6833	0.6819	-0.0014	58m38s	66%	
luna6_n20-17	32	6	0.6841	0.6817	-0.0024	58m49s	64%	
luna6_n20-18	36	10	0.6849	0.6831	-0.0018	58m44s	63%	keep_rule
luna6_n20-19	28	11	0.6826	0.6811	-0.0015	58m45s	65%	
luna6_n20-20	38	10	0.6850	0.6844	-0.0006	58m52s	64%	

B Validity checks in detail

Table 10: The runs with an integrity flag, the leak check included (all false alarms on review), or a protocol caveat. No run was excluded.

run	integrity flag (cleared)	caveat	resolution
astra6_n20-4	artifact_outside_clock		explained: the last experiment, started inside the clock with 1m53s left, was cut off during evaluation when the turn failed
astra6_n20-5		keep_rule	7b7712f (gamma 5) kept at a tie as “faster” by 0.8 s, within noise, not simpler; it stayed in the final model
astra6_n20-6	leak_check_hit		false match: the leak check matched the end of a column list that the agent printed in its own setup check
astra6_n20-8	leak_check_hit	keep_rule	false match: the leak check matched a call to the harness’s own artifact loader in the agent’s final check; e5e4a2b (max_cat_threshold 16) kept at a tie as “faster” by 0.2 s, within noise, not simpler; it stayed in the final model
astra6_n20-12		keep_rule	d4bd0fc (max_bin 64) kept at a tie as “faster” by 1.1 s in one timing, within noise, not simpler; it stayed in the final model
astra6_n20-15	train_py_review		explained: a read of data/train.csv written differently from the starter, in a discarded commit
astra6_n20-16	artifact_outside_clock		explained: the agent reset the repository before the evaluation of a discarded commit had finished, so its timing row carries the wrong commit

run	integrity flag (cleared)	caveat	resolution
sol6_n20-8		keep_rule	83775c5 (DART <code>rate_drop</code> 0.2) kept at a tie as “faster” by 1.0 s, within noise, not simpler; it is the final model
sol6_n20-12	train_py_review		false match: the check’s pattern <code>glob</code> matches the word <code>global</code> in a variable name (e.g. <code>global_target_rate</code>)
sol6_n20-16		turn_retries	7 failed turns, all ‘model at capacity’; retry waits of 216 s
sol6_n20-19		keep_rule	f833740 (lossguide tree growth) kept at a tie as “faster” by 1.0 s, within noise, not simpler; lossguide stayed in the final model
luna6_n20-1	train_py_review		false match: the check’s pattern <code>glob</code> matches the word <code>global</code> in a variable name (e.g. <code>global_target_rate</code>)
luna6_n20-6		keep_rule	bd192f4 (gamma 0.1) kept at a tie as “faster” by 0.6 s, within noise, not simpler; it changed neither the eval nor the holdout AUC
luna6_n20-7	train_py_review		false match: the check’s pattern <code>glob</code> matches the word <code>global</code> in a variable name (e.g. <code>global_target_rate</code>)
luna6_n20-8		keep_rule	744ceef and 7a1cbf0 (<code>min_child_weight</code> 5 and 10) kept at ties as “faster”, within noise, not simpler; neither changed the eval or the holdout AUC
luna6_n20-11	train_py_review		false match: the check’s pattern <code>glob</code> matches the word <code>global</code> in a variable name (e.g. <code>global_target_rate</code>)
luna6_n20-15	train_py_review	keep_rule	false match: the check’s pattern <code>glob</code> matches the word <code>global</code> in a variable name (e.g. <code>global_target_rate</code>); 744705b and 41db4a7 (gamma 1 and 2) kept at ties as “faster”, within noise, not simpler; neither changed the eval or the holdout AUC, and gamma 2 stayed in the final model
luna6_n20-18		keep_rule	dfccd9c (gamma 1) kept at a tie as “faster” by 0.3 s, within noise, not simpler; it changed neither the eval nor the holdout AUC (the run also needed a second “go”)

Table 11: The headline statistics without the runs with a protocol caveat and without the runs that used BTS documentation about the evaluation year (astra6_n20-7, astra6_n20-12 and sol6_n20-11).

runs	n (Astra, Sol, Luna)	mean Astra	mean Sol	mean Luna	P(Astra beats Sol)	P(Astra beats Luna)	P(Sol beats Luna)
all 60 runs	20, 20, 20	0.6866	0.6842	0.6806	80%	98%	91%
without the 10 caveat runs	17, 17, 16	0.6867	0.6841	0.6807	82%	97%	91%
without the 3 BTS-informed runs	18, 19, 20	0.6864	0.6841	0.6806	79%	98%	91%

C The rules given to the agent

Below are the sections of `program.md` (xgboost-autoresearch-minimal3 at commit 5fb023a) that define the task, what the agent may and may not do, the keep rule, the experiment loop and the time budget, reproduced verbatim. The file also contains setup steps, the research requirements, the feature-engineering rules for row-local features, the output and logging formats, and the timeout and crash rules.

The task: predict whether a flight departs 15 or more minutes late (`dep_delayed_15min`, Y/N) from what is known in advance: month, day of month, day of week, scheduled departure time, carrier, origin, destination and distance. The metric is AUC. `data/train.csv` has 200K rows sampled from 2005 flights and `data/eval.csv` has 50K rows from 2006; both are balanced (half delayed, half not). You optimize the AUC on `eval.csv`; after the run the human checks your models on a holdout set that you never see.

Experimentation

You launch an experiment as: `python3 harness.py run`. It runs `train.py`, times it, kills it if it exceeds the time limits (1 minute for training, 5 minutes for evaluation, see **Timeout**), and refuses to start once the time budget is used up. Do not run `python3 train.py` directly while the clock is running (it will refuse once training is done, before evaluating).

What you CAN do:

- Modify `train.py` - this is the only code file you edit (besides your own `output/results.tsv` and `output/research-log.md`). Everything is fair game that will lead to a model that generalizes on unseen data: data preparation, feature engineering, choosing hyperparameters, and model training. You can also implement new features such as early stopping etc.
- Inspect `data/train.csv` with read-only scratch commands (e.g. `python3 -c ...`) to understand the data. Do not create other code files; everything that affects the model stays in `train.py`.
- Search the web and read external resources. This is not optional — you **MUST** do research before relying solely on your own intuition. See the **Research** section below.

What you CANNOT do:

- Do not read, run, or modify anything in the `human/` folder: it holds the human-only tools. `human/make_data.py` builds the data splits, including the holdout set.
- Do not install new packages or add dependencies. You can only use what's already installed.
- Do not change the evaluation. The final model is trained on `train.csv` and evaluated by the call `save_and_evaluate(model, prepare)` at the end of `train.py`, which must stay the last line. Do not add cross-validation, a retrain on more data, or any other evaluation of your own as the metric.
- Do not modify the evaluation harness. Do not modify `harness.py`, and do not modify or delete anything in `artifacts/` or `output/timing/`.
- Do not use any of the data files other than `train.csv`. Only `data/train.csv` may

be read in `train.py`; `data/eval.csv` is read by `harness.py` for evaluation only. Never read, open, or inspect `data/holdout.csv` or the source data `2005.csv` and `2006.csv` (the latter contains the holdout rows) in any way, wherever they are stored. If you need a validation set (e.g. for early stopping), split it off `train.csv`. Note that `train.csv` is sampled from 2005 flights while `eval.csv` (and the holdout set) is from 2006, so a validation split off `train.csv` overstates the AUC and can favour more complex models than the eval set does.

- Do not read, run, or reference `human/score_holdout.py`, `human/score_holdout_all.sh` or `human/plot_auc_history.py`, and do not read their outputs `output/holdout_scores.tsv` and `output/auc_history.png`. These are human-only tools for post-hoc evaluation of experiments against the holdout set. They are never part of the experiment loop. If you find yourself wanting to use them, stop and tell the human immediately — it means something has gone wrong with your understanding of the task.
- Do not use git to peek at the results of earlier runs: do not look in the git history for `results.tsv`, `holdout_scores.tsv` or any other `.tsv`, `.txt` or `.png` file with results.
- Do not peek into results in the `results` folder and its sub-folders (archived earlier runs, including their holdout scores).

Keep rule: Keep an experiment only if its Eval AUC, as printed by the harness (4 decimals), is higher than that of the last kept commit. If it is exactly equal, keep it only if the code is simpler or faster (a simplification or clean-up at no cost in AUC). If it is lower, discard it, however small the drop and however much simpler the code. All else being equal, simpler is better: removing something and getting an equal or better result is a great outcome.

The experiment loop

The experiment runs on a dedicated branch (e.g. `mar5`).

LOOP until the time budget is used up:

1. Check the clock: `python3 harness.py status`. If it prints `TIME IS UP`, or less than 2 minutes remain, stop the loop and wrap up (see **Time budget**). Otherwise look at the git state: the current branch/commit we're on
2. **Choose your next experiment deliberately.** Before touching any code:
 - Review `output/results.tsv` and recent commits.
 - State a short **hypothesis**: what you are changing, why you think it will help, and (if applicable) which prior result motivates this step.
 - Classify the experiment as one of: *follow-up* to a promising result, *ablation/simplification* of a promising result, or *exploration* of a meaningfully different direction.
 - **Do not** run near-duplicate experiments unless you can state exactly what is different and why it matters. Avoid random-walk behavior and cosmetic variations of the same idea.
 - If you haven't done web research in the last 10 experiments, or if you hit a plateau (3+ consecutive discards with <0.001 movement), do research now before proposing your next change. See the **Research** section.
3. Tune `train.py` with that experimental idea by directly hacking the code.

4. git commit `train.py` only (everything in `output/` stays uncommitted; the human archives or deletes it after the run)
5. Run the experiment: `python3 harness.py run > output/run.log 2>&1` (redirect everything - do NOT use tee or let output flood your context). Run one experiment at a time.
6. Read out the results: `grep "^Eval AUC:" output/run.log`
7. If the grep output is empty, the run crashed or timed out (or the time budget is used up). Run `tail -n 50 output/run.log` to read the Python stack trace and attempt a fix. If you can't get things to work after more than a few attempts, give up on that idea and move on.
8. Record the results in the tsv (NOTE: do not commit the results.tsv file, leave it untracked by git)
9. If Eval AUC improved (higher), or is exactly equal with simpler or faster code (see the **Keep rule**), you "advance" the branch, keeping the git commit
10. Otherwise (Eval AUC lower, or equal without a simplification), you discard it: `git reset --hard <hash of the last kept commit>` (the untracked `output/` folder is not affected), then check with `git log --oneline -1` that HEAD is that last kept commit
11. **Every 10 experiments** (not counting the baseline), pause and briefly synthesize what you have learned so far: what kinds of changes help, what kinds do not, what your current best theory is about what matters on this dataset, and what direction to try next. Write this synthesis as a short section in `output/research-log.md` to inform subsequent experiments.

The idea is that you are a completely autonomous researcher trying things out. If they work, keep. If they don't, discard. And you're advancing the branch so that you can iterate. If you feel like you're getting stuck in some way, you can rewind but you should probably do this very very sparingly (if ever).

Time budget: You have 1 hour of wall-clock time from `python3 harness.py start`, counting everything: thinking, research, editing and runs. Within the budget, do NOT pause to ask the human if you should continue. Do NOT ask "should I keep going?" or "is this a good stopping point?". The human might be asleep, or gone from a computer and expects you to keep working until the budget is used up. You are autonomous. If you run out of ideas, think harder - read papers and documentation, re-read the in-scope files for new angles, try combining previous near-misses, try more radical changes to the features or the model setup. Do not wrap up or stop the clock while 2 minutes or more remain.

D Statistical details

Table 12: Two-sided tests between LLMs and, post hoc, the runs per arm, rounded up, that the approximation $16 \text{ sd}^2 / \text{gap}^2$ gives for 80% power at the 5% level with the observed gap and pooled sd. One-way ANOVA: $F = 45.3$, $p = 2\text{e-}12$; Kruskal-Wallis: $H = 37.0$, $p = 9\text{e-}09$.

pair	Welch t	p	Mann-Whitney U	p	runs per arm to detect the observed gap
Astra vs Sol	3.7	6e-04	320	1e-03	12

pair	Welch t	p	Mann-Whitney U	p	runs per arm to detect the observed gap
Astra vs Luna	9.2	4e-11	391	3e-07	2
Sol vs Luna	6.0	6e-07	364	9e-06	5

The runs-per-arm column applies the approximation $16 \text{ sd}^2 / \text{gap}^2$, which gives the number of runs per arm for 80% power in a two-sided test at the 5% level with sd the run-to-run standard deviation and gap the difference between the means (Miller, 2024; Colas et al., 2018). The gaps were observed, not fixed in advance, so these counts illustrate the scale rather than prescribe a design: on this task, telling the two closest LLMs apart takes about twelve runs of each, and a comparison fixed in advance should be sized on the smallest difference that would change a decision.

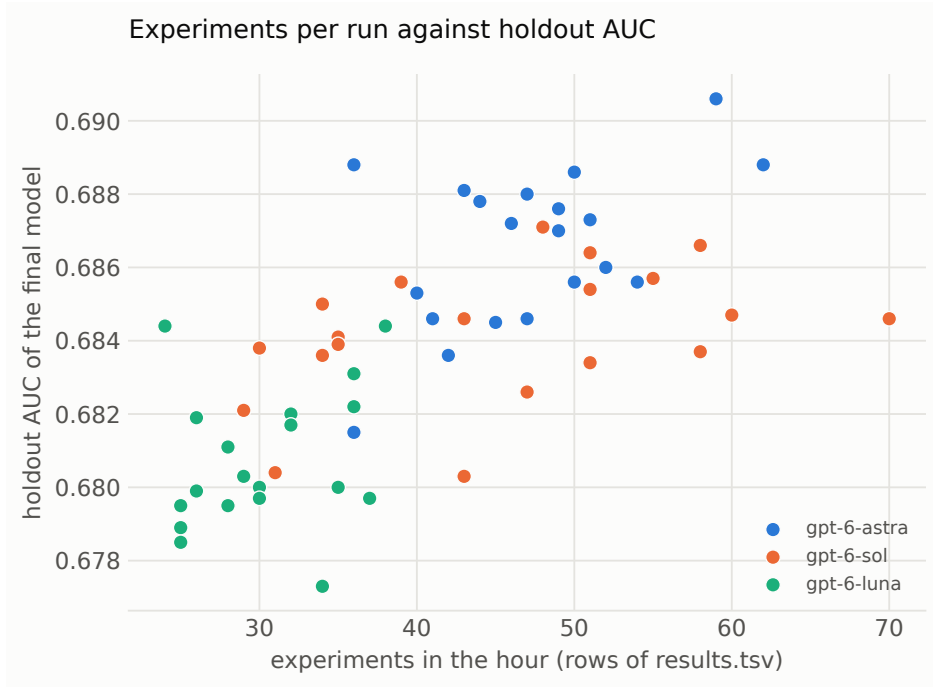


Figure 8: Experiments per run against the holdout AUC of the final model, by LLM. The rank correlation within an LLM is 0.36 to 0.45; exploratory.

E The feature audit

A script reads the final `train.py` of every run and records how it handles the calendar and what model it fits: whether `DayOfMonth` and `Month` are still in the list of categorical columns, whether either is converted to a number, whether the file builds holiday features (holiday, Thanksgiving, Christmas and similar words outside comments) or day-of-year-like features (day of year, week or fortnight of the year), the tree depth or leaf limit, the number of trees and the learning rate, and whether more than one model is fitted or averaged. The script also flags files whose structure it may misread: a day-of-month column both kept as a category and converted to a number, a category list that is not a single plain assignment, holiday words only in comments, or a use of the column inside `prepare` that it does not recognize; ten of the 60 files carry such a flag. Because the script

matches names and patterns, every file was then read in full. Written rules fix what counts. Only what reaches the fitted models counts, so a day-of-year value used only to compute holiday offsets is not a day-of-year-like feature; a numeric month, or a sine and cosine of the month, is not one either; and a single booster with several parallel trees is one model, not an ensemble. Three Claude agents read 20 files each and recorded every verdict with the line numbers it rests on. The Claude session that orchestrated the study then read the ten flagged files and every file where a reader’s calendar verdict differed from the script’s, and settled the verdicts. The rules, the readers’ verdicts with their evidence, and the settled verdicts are in the paper’s repository.

The reading confirmed the script for 40 files and corrected 20. Every day-of-month verdict, on which the calendar finding rests, was confirmed, including the eight files that also convert the day of the month to a number: in all of them the category reaches the model, and the number only feeds a day-of-year or holiday feature. The calendar corrections are eight Astra files whose day-of-year variable feeds only holiday features capped at three weeks or less, one Sol file that keeps the month as a category under a new name, and one Sol file with a year-end travel window; the other corrections concern ensembles and the settings of their members. Table 13 gives the per-run result, and Table 6 and Figure 5 use the settled verdicts.

Table 13: The final model of every run. A script read each final train.py; every file was then read in full, and the script’s verdicts were confirmed or corrected (Appendix E). Depth, trees and learning rate as read for the models of the final prediction, each distinct value once, in the order of the file; “16 leaves” is a leaf limit for lossguide trees. Ensemble: the prediction combines more than one fitted model (different configurations, seeds or a blend).

run	hold-out AUC	day-of-month cate- gory	month cate- gory	day-of-year- like	holiday features	depth	trees	learning rate	ensem- ble	verified
astra6_n20-1	0.6880				yes	16 leaves	400	0.025	yes	confirmed
astra6_n20-2	0.6836	yes	yes			6	1600	0.05		corrected
astra6_n20-3	0.6886				yes	32 leaves	600	0.04	yes	corrected
astra6_n20-4	0.6815		yes		yes	5	1000	0.02		corrected
astra6_n20-5	0.6846				yes	3	300	0.05		corrected
astra6_n20-6	0.6846		yes			16 leaves	500	0.05		corrected
astra6_n20-7	0.6881				yes	3, 7	200, 500	0.1, 0.05	yes	confirmed
astra6_n20-8	0.6856				yes	24 leaves	100	0.05		corrected
astra6_n20-9	0.6888		yes		yes	2, 32 leaves	200, 600	0.05	yes	confirmed
astra6_n20-10	0.6845		yes		yes	8	400	0.05		corrected
astra6_n20-11	0.6860				yes	4 leaves, 8 leaves, 16 leaves	500, 300, 200	0.05	yes	corrected
astra6_n20-12	0.6873				yes	3	500	0.05		corrected
astra6_n20-13	0.6906			yes	yes	16 leaves	600	0.05	yes	confirmed
astra6_n20-14	0.6888				yes	3, 2, 4, 5	600, 1200, 300, 150	0.05	yes	corrected
astra6_n20-15	0.6856		yes	yes		4, 3, 5	200, 400, 100	0.08	yes	corrected
astra6_n20-16	0.6870				yes	4	180	0.1	yes	corrected
astra6_n20-17	0.6872			yes		4	500	0.05	yes	confirmed
astra6_n20-18	0.6878		yes		yes	16 leaves	1200	0.05	yes	corrected
astra6_n20-19	0.6876		yes		yes	4	300	0.05	yes	corrected

run	hold- out AUC	day-of- month cate- gory	month cate- gory	day- of- year- like	holiday features	depth	trees	learning rate	ensem- ble	verified
astra6_n20-20	0.6853					32 leaves	600	0.05		corrected
sol6_n20-1	0.6850			yes		3, 4	150, 100	0.1	yes	confirmed
sol6_n20-2	0.6836	yes	yes	yes		4, 3	200	0.05	yes	confirmed
sol6_n20-3	0.6841		yes	yes		3	300	0.05		confirmed
sol6_n20-4	0.6821	yes	yes	yes		3	200	0.05		confirmed
sol6_n20-5	0.6826	yes	yes	yes	yes	6	275	0.025	yes	confirmed
sol6_n20-6	0.6803	yes	yes	yes		31 leaves (depth 10)	120	0.05		confirmed
sol6_n20-7	0.6837		yes			3, 4	130, 160	0.1	yes	confirmed
sol6_n20-8	0.6871				yes	3, 4	300, 200	0.05	yes	corrected
sol6_n20-9	0.6834	yes	yes	yes		3	400	0.05	yes	confirmed
sol6_n20-10	0.6864					3	400	0.05	yes	confirmed
sol6_n20-11	0.6857			yes	yes	3	500	0.05		confirmed
sol6_n20-12	0.6846		yes	yes		3	150	0.1		corrected
sol6_n20-13	0.6854			yes		3, 2, 4	400, 700, 18	0.05	yes	confirmed
sol6_n20-14	0.6846			yes		3, 4	100	0.1	yes	confirmed
sol6_n20-15	0.6856			yes		16 leaves	200	0.05		confirmed
sol6_n20-16	0.6804	yes	yes			4	400	0.05		confirmed
sol6_n20-17	0.6839	yes	yes	yes	yes	3, 4	250	0.05	yes	confirmed
sol6_n20-18	0.6847		yes	yes		3, 4	300	0.05	yes	corrected
sol6_n20-19	0.6866			yes		8 leaves, 12 leaves	500	0.05	yes	corrected
sol6_n20-20	0.6838		yes	yes		3	300	0.05		confirmed
luna6_n20-1	0.6822	yes	yes			6	800	0.02		confirmed
luna6_n20-2	0.6789	yes	yes			5	300	0.05		confirmed
luna6_n20-3	0.6803	yes	yes			4	200	0.05	yes	confirmed
luna6_n20-4	0.6797	yes	yes			4	300	0.03		confirmed
luna6_n20-5	0.6789	yes	yes			4	200	0.05		confirmed
luna6_n20-6	0.6800	yes	yes			3	100	0.1		confirmed
luna6_n20-7	0.6795	yes	yes			4	200	0.05		confirmed
luna6_n20-8	0.6795	yes	yes			3	200	0.05		confirmed
luna6_n20-9	0.6844			yes		4	200	0.05		confirmed
luna6_n20-10	0.6820	yes				16 leaves	200	0.05		confirmed
luna6_n20-11	0.6773	yes	yes			6	100	0.1		confirmed
luna6_n20-12	0.6799	yes	yes			6	450	0.1		confirmed
luna6_n20-13	0.6797	yes	yes		yes	5	200	0.05		confirmed
luna6_n20-14	0.6800	yes	yes			3	300	0.05		confirmed
luna6_n20-15	0.6785	yes	yes			4	100	0.1		confirmed
luna6_n20-16	0.6819	yes	yes		yes	4	200	0.05		confirmed
luna6_n20-17	0.6817			yes		3	100	0.1		confirmed
luna6_n20-18	0.6831			yes		3	200	0.1		confirmed
luna6_n20-19	0.6811	yes	yes	yes		3, 4	800	0.0125	yes	corrected
luna6_n20-20	0.6844			yes		4	200	0.05		confirmed

F Software, machines, dates and disclosure

Software in the containers: Codex CLI 0.160.0; Python 3 with xgboost 3.4.1, pandas 3.0.6, scikit-learn 1.9.1, numpy 2.5.3 and cloudpickle 3.1.2; Ubuntu 26.04. The single-run repository (Pafka, 2026a) at commit 5fb023a, the version every run used; the orchestrator, the archived runs and the plotting tools (Pafka, 2026b) at commit c34a051, which differs from b2841ab, the commit at which the runs were archived, only by the license; the analysis scripts of this paper at github.com/szilard/xgboost-autoresearch3-paper. All three repositories are released under the MIT license. A blog post with the main results preceded this paper (Pafka and Ariño de la Rubia, 2026b). The flight data are public (Data Expo 2009); `human/make_data.py` rebuilds the three files from them with fixed seeds. The final `train.py` of every run is archived; the model artifacts were deleted with the containers.

Table 14: Operational summary. Failed turns ended with the service error ‘model at capacity’ and were retried; the clock kept running. Every run’s agent stopped the clock itself; the clock could exceed the hour when the agent’s wrap-up came after its last status check. Nothing was killed at the 24 GiB memory cap.

LLM	dates (UTC)	failed turns	runs with retry waits	runs stopped after the budget	clock, min:s, shortest to longest	runs with a context compaction	peak memory, GiB
Astra	2026-10-06 to 2026-10-07	1	1	4	58:42–61:18	11	3.4–16.5
Sol	2026-10-05 to 2026-10-06	13	4	2	58:26–60:59	2	3.6–11.7
Luna	2026-10-05 to 2026-10-06	0	0	0	58:31–59:42	12	3.6–12.0

Claude Code, running the orchestrator’s project skill, launched every run through the fixed driver script, reviewed each finished run against the checks and drafted its review file; it also read every final `train.py` for the feature audit (Appendix E) and helped write the tools, the blog post that preceded this paper, and this paper. Codex, the agent under study, did all the modeling in every run. The authors designed the study, reviewed every run’s review, and are responsible for the text and the numbers.

G One run in detail

Figure 9 and Table 15 show one Astra run, `astra6_n20-1`, chosen because it is cleanly documented and lies above the median of its LLM (holdout AUC 0.6880, sixth of Astra’s 20 runs).

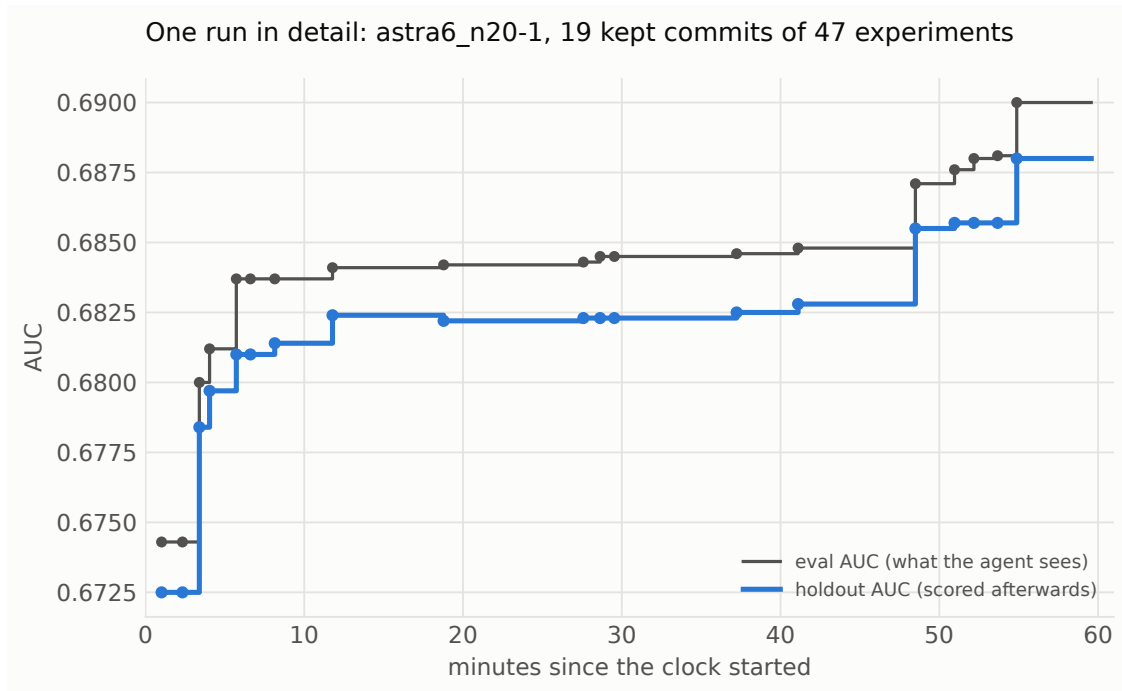


Figure 9: One run in detail: eval AUC (gray) and holdout AUC (blue) of each kept commit of run astra6_n20-1 over the hour.

Table 15: The baseline and the 18 kept changes of run astra6_n20-1, in order, with the minute of the clock at which each was kept.

minute	the agent's description of the kept commit	eval AUC	holdout AUC
1	baseline	0.6743	0.6725
2	simplify prepare; same features and faster evaluation	0.6743	0.6725
3	depth 4; 400 trees; stronger leaf and L2 regularization	0.6800	0.6784
4	remove day-of-month calendar feature	0.6812	0.6797
6	remove month; retain weekday and schedule inputs	0.6837	0.6810
7	fixed category codes; identical AUC and faster evaluation	0.6837	0.6810
8	200 rounds; equal AUC with a smaller faster model	0.6837	0.6814
12	departure offsets from train-fitted origin/route/carrier-origin medians	0.6841	0.6824
19	balance class weights within each training month	0.6842	0.6822
28	equal blend of schedule and geography classifiers	0.6843	0.6823
29	three fixed seeds for each schedule/geography representation	0.6845	0.6823
30	remove geography; equal AUC with half the ensemble	0.6845	0.6823
37	400 rounds at learning rate 0.025	0.6846	0.6825
41	lossguide growth with 16 leaves	0.6848	0.6828
48	narrow Thanksgiving/winter/July-4 relative-date features	0.6871	0.6855
51	add Memorial Day and Labor Day offsets	0.6876	0.6857
52	shared MLK/Presidents Day relative-date feature	0.6880	0.6857
54	remove route-time offset from holiday ensemble	0.6881	0.6857
55	remove monthly weighting after adding holiday features	0.6900	0.6880

H Token usage

Table 16 summarizes the token usage per run, from the cumulative usage records in the session logs. Astra made far fewer API responses per run than the other two LLMs (126 against about 300) and used less than half their input tokens; for all three LLMs, 98 to 99% of the input tokens were served from the cache. The last column projects list prices and is not a bill, since the runs ran on a subscription.

Table 16: Token usage per run, means over the 20 runs, from the cumulative usage records in the session logs. The projection applies OpenAI’s list prices per million tokens (input / cached input / output: Astra 10 / 1 / 50, Sol 2 / 0.20 / 10, Luna 0.10 / 0.01 / 0.50) and is not a bill: the runs ran on a ChatGPT subscription.

LLM	input tokens, millions (min–max)	cached share	output tokens, thousands	of which reasoning	API responses	list-price projection, USD (min–max)
Astra	16.0 (12.2–21.4)	98.1%	89	50	126	23.17 (18.79–29.01)
Sol	34.6 (21.9–51.8)	99.1%	65	25	304	8.11 (5.37–11.69)
Luna	37.1 (22.6–55.2)	98.6%	104	65	298	0.47 (0.31–0.65)